

DSWD REGION X

WGIMS

Welfare Goods Inventory Management System
Complete Process Flow & System Workflow Documentation

Subsidies • Deliveries • RIS Requisitions • Stock Transfers • Stock Cards • Reports • Audit

Based on the Laravel source code · August 25, 2026

Table of Contents

PART 1 — HIGH-LEVEL OVERALL SYSTEM FLOW

- 1.1 The big picture: how goods flow through WGIMS
- 1.2 Overall system process flow (flowchart)
- 1.3 Module relationship map (how documents link together)

PART 2 — USER LOGIN & AUTHENTICATION

- 2.1 Login flowchart
- 2.2 What happens on every request after login
- 2.3 Logout & session protection
- 2.4 Files behind each step

PART 3 — ITEM & ITEM CATEGORY MANAGEMENT (Master Data)

- 3.1 Flowchart
- 3.2 How stock records (Items) relate to the catalog
- 3.3 Files behind each step

PART 4 — SUPPLIER / SUBSIDY / DELIVERY FLOW (detailed)

- 4.1 Stage 1 — Create the Subsidy (Admin or Warehouse Manager)
- 4.2 Stage 2 — Record a Delivery / Dispatch batch (Admin or Warehouse Manager)
- 4.3 Stage 3 — Corrections and reversals (Admin only)
- 4.4 Files behind each step

PART 5 — REQUISITIONS / RIS FLOW (detailed)

- 5.1 Stage 1 — Create the RIS (Admin or Warehouse Manager)
- 5.2 Stage 2 — Approve & Issue (Admin, Warehouse Manager, Supply Custodian, Center Head — NOT Center Staff)
- 5.3 Stage 3 — Corrections and reversals (Admin only)
- 5.4 Files behind each step

PART 6 — STOCK TRANSFER FLOW (detailed)

- 6.1 Stage 1 — Create transfer request (Admin or Warehouse Manager)
- 6.2 Stage 2 — Dispatch (Admin or Warehouse Manager)
- 6.3 Stage 3 — Correction / Deletion (Admin only)
- 6.4 Files behind each step

PART 7 — INVENTORY / STOCK MANAGEMENT & STOCK CARDS (detailed)

- 7.1 What a "stock record" is (the heart of the system)
- 7.2 Quantity movement map — every way stock changes
- 7.3 Stock Card engine (automatic ledger)
- 7.4 Inventory Balance report (live) (Admin + Warehouse Manager only)
- 7.5 Files behind each step

PART 8 — REPORTS

- 8.1 Report flows
- 8.2 Files behind each step

PART 9 — AUDIT & DATA INTEGRITY

- 9.1 Three append-only audit trails

9.2 Data-integrity rules the system enforces (QA checklist)

APPENDIX — Quick file index by module

Purpose of this document

- Understand how the entire system works end-to-end
- Use as a testing/QA checklist (every decision point = a test case)
- Locate where bugs occur (each box names the exact file that runs it)
- Explain the system to users and support the user manual
- Show exactly how inventory quantities move through the system

How to read these diagrams

- **Diamond shapes** are decision points (questions the system asks).
- **Rectangles** are actions/steps.
- **Text in brackets** like `[AuthController.php:41]` tells you the exact file and line that performs the step — useful for QA and debugging.
- **Bold role names** show who is allowed to do each step.

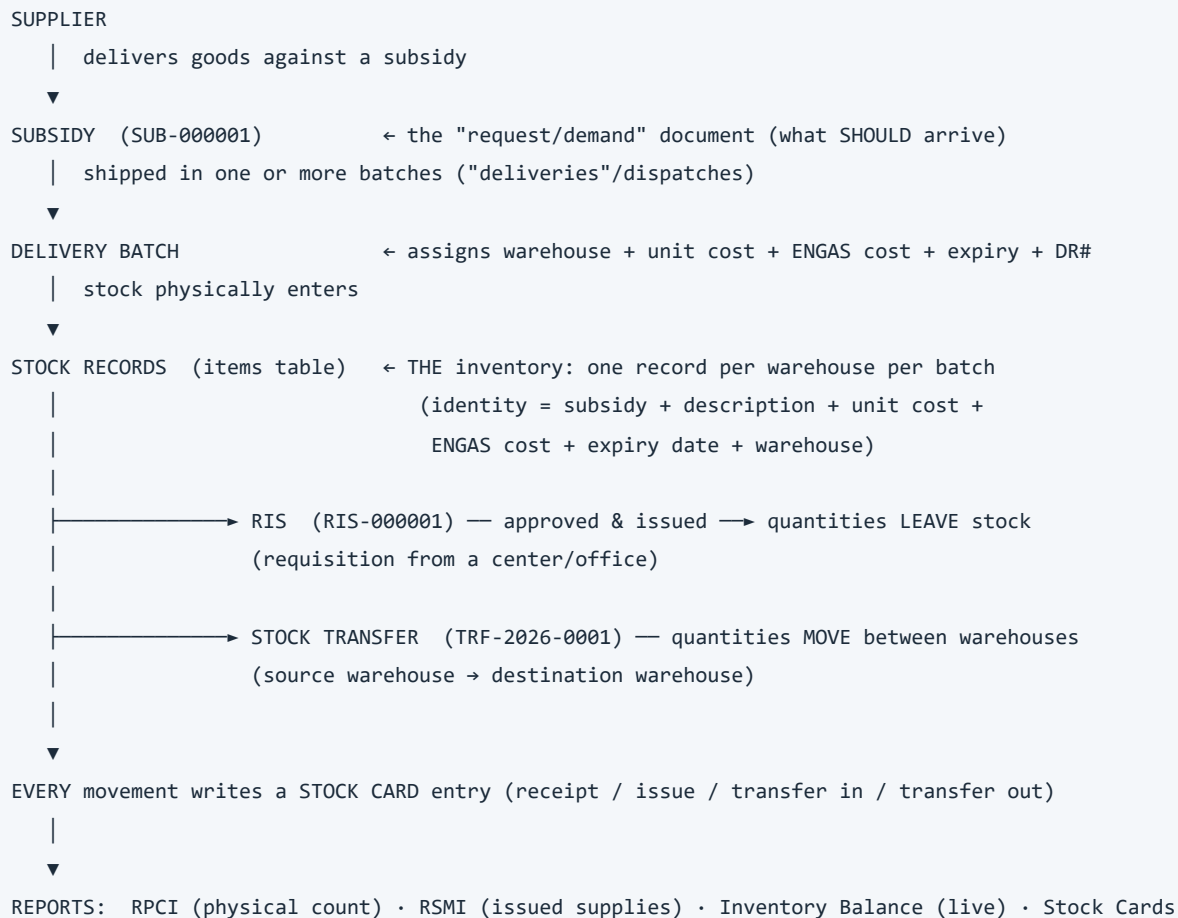
The 5 user roles and what they may do

Role	Create documents (subsidy, delivery, RIS, transfer)	Approve/issue RIS	Edit master data (categories, suppliers, warehouses, users)	Correct / delete documents
Admin	✓	✓	✓	✓
Warehouse Manager	✓	✓	✗ (view only)	✗
Supply Custodian	✗	✓	✗	✗
Center Head	✗	✓	✗	✗
Center Staff	✗	✗	✗	✗

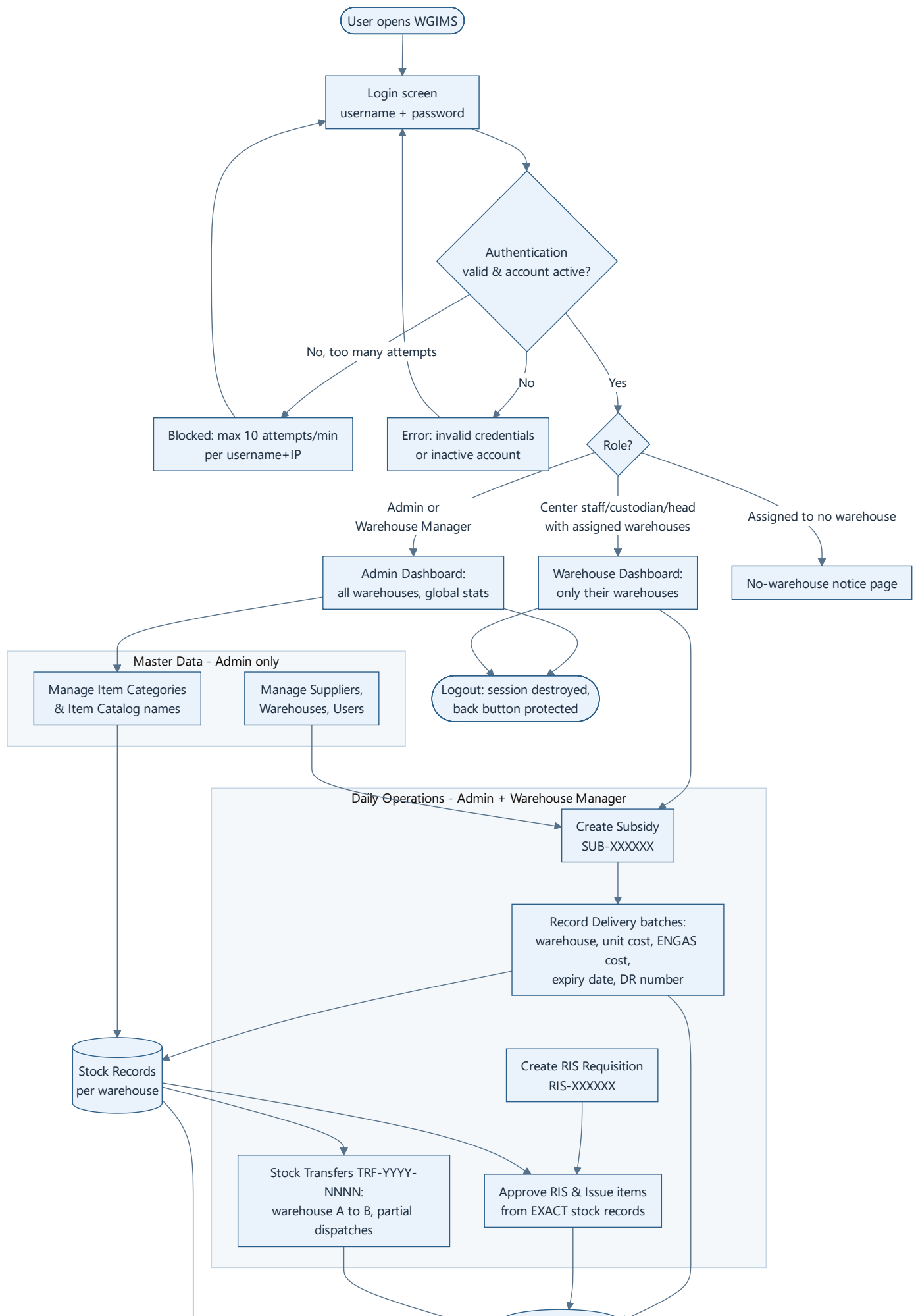
All non-admin/WM roles see **only the warehouses assigned to them** (`app/Http/Controllers/Concerns/ScopesWarehouse.php`).

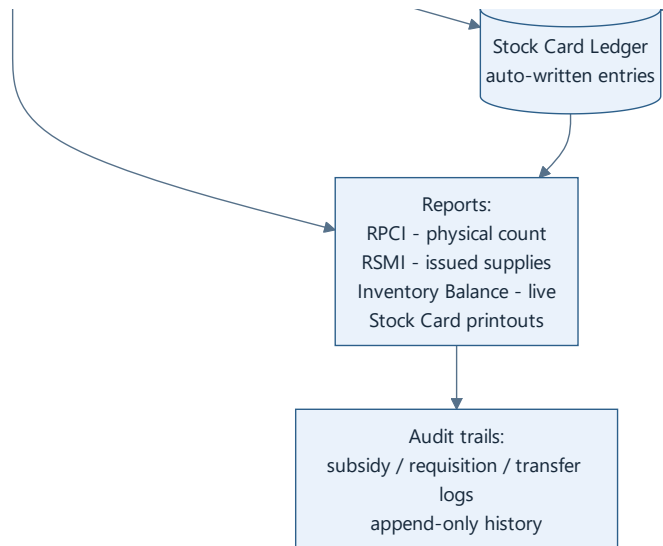
PART 1 — HIGH-LEVEL OVERALL SYSTEM FLOW

1.1 The big picture: how goods flow through WGIMS

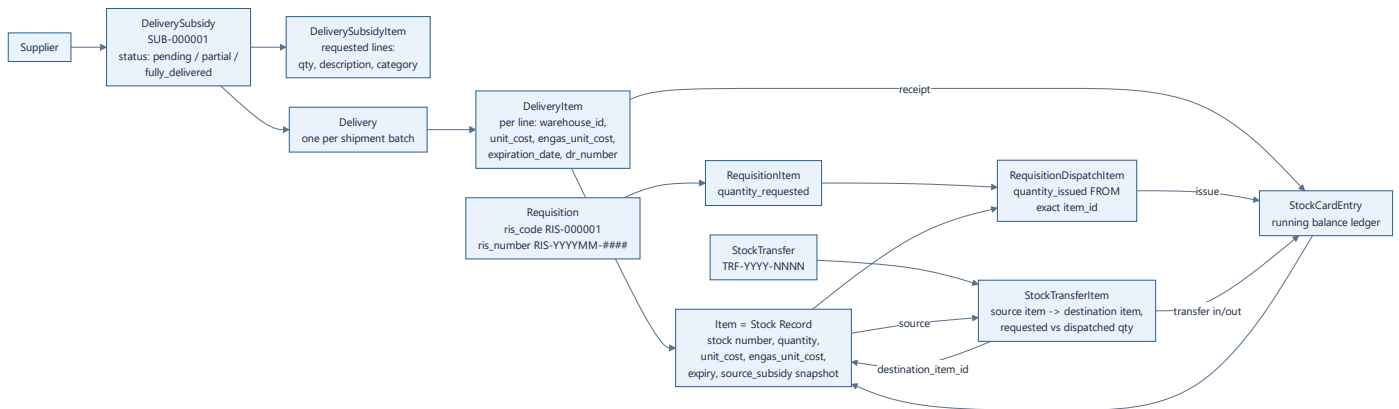


1.2 Overall system process flow (flowchart)





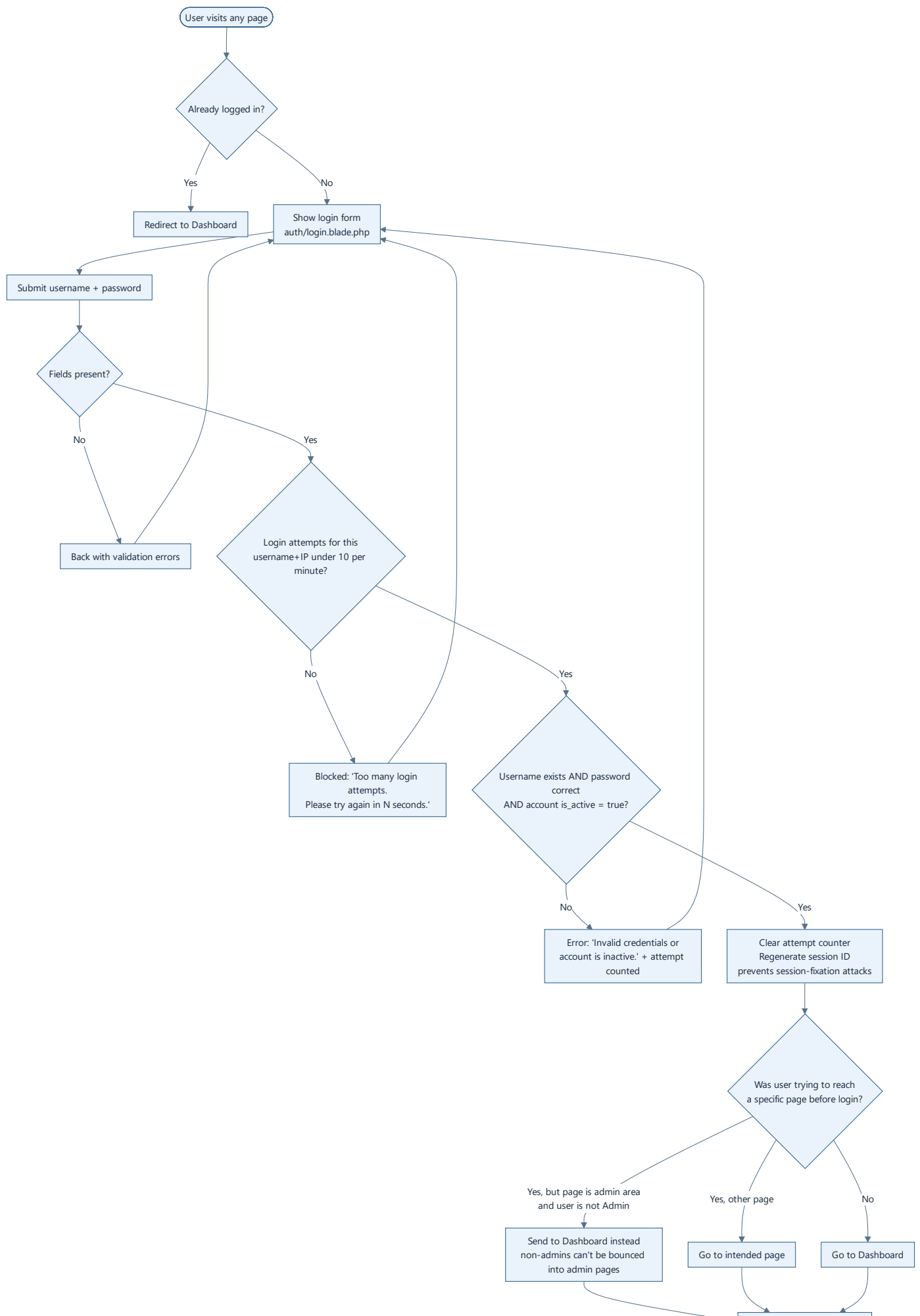
1.3 Module relationship map (how documents link together)



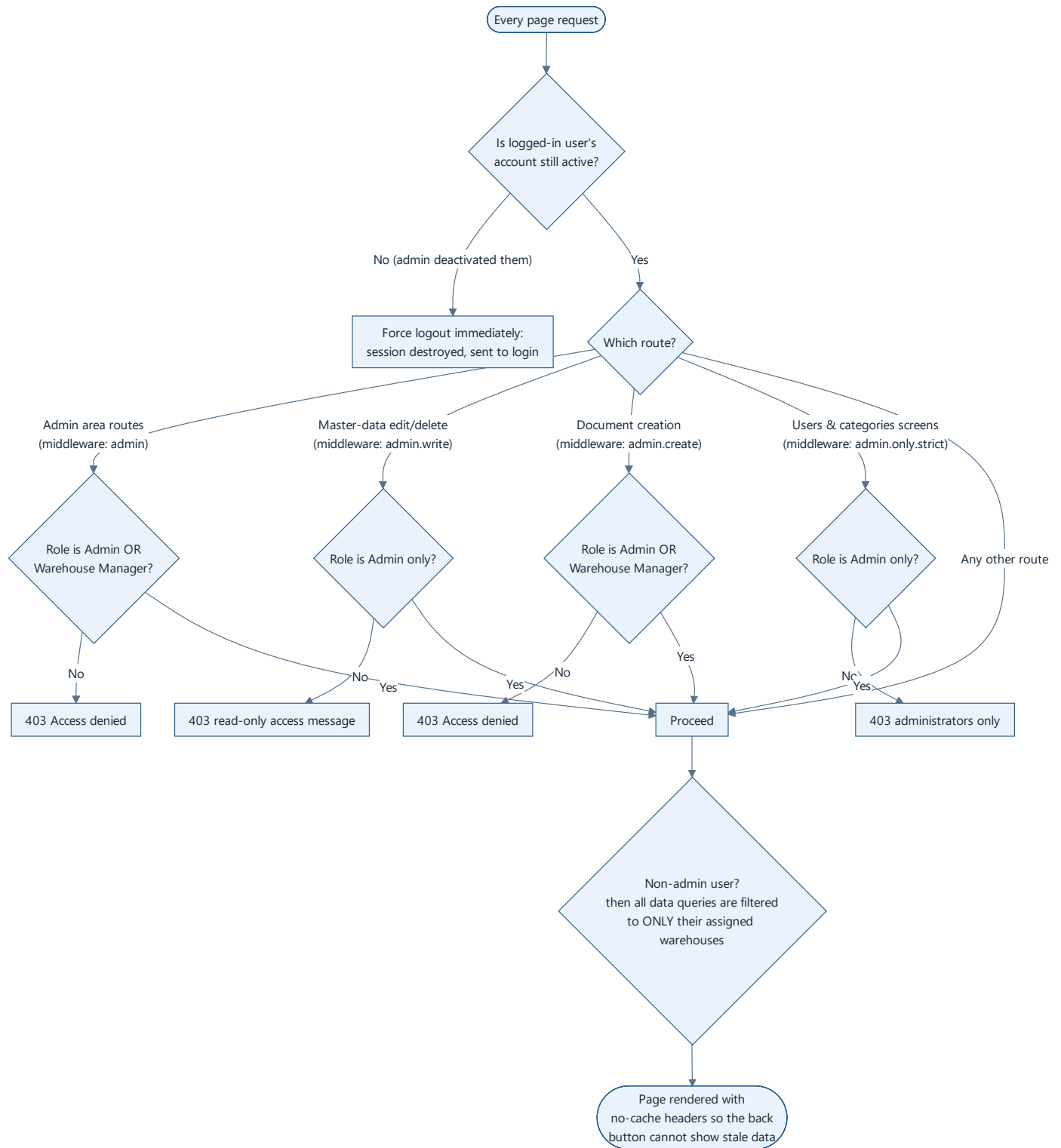
Key idea for non-programmers: every quantity in WGIMS lives on an *Item row* (a stock record). Documents never hold stock themselves — they only move quantities onto or off of stock records, and every movement is journalled into a Stock Card entry so the running balance can always be replayed and verified.

PART 2 — USER LOGIN & AUTHENTICATION

2.1 Login flowchart



2.2 What happens on every request after login



2.3 Logout & session protection

POST Logout

```
graph TD; A([POST Logout]) --> B[Destroy login session]; B --> C[Regenerate CSRF token]; C --> D[Instruct browser to clear cached site data Clear-Site-Data header]; D --> E[ ];
```

The flowchart illustrates the POST Logout process. It begins with an oval-shaped start node labeled 'POST Logout'. An arrow points down to a rectangular process node labeled 'Destroy login session'. Another arrow points down to a second rectangular process node labeled 'Regenerate CSRF token'. A third arrow points down to a larger rectangular process node labeled 'Instruct browser to clear cached site data Clear-Site-Data header'. Finally, an arrow points down from the bottom of this last node, indicating the process continues.

Destroy login session

Regenerate CSRF token

Instruct browser to clear cached
site data Clear-Site-Data header

Back to login form

Protection summary:

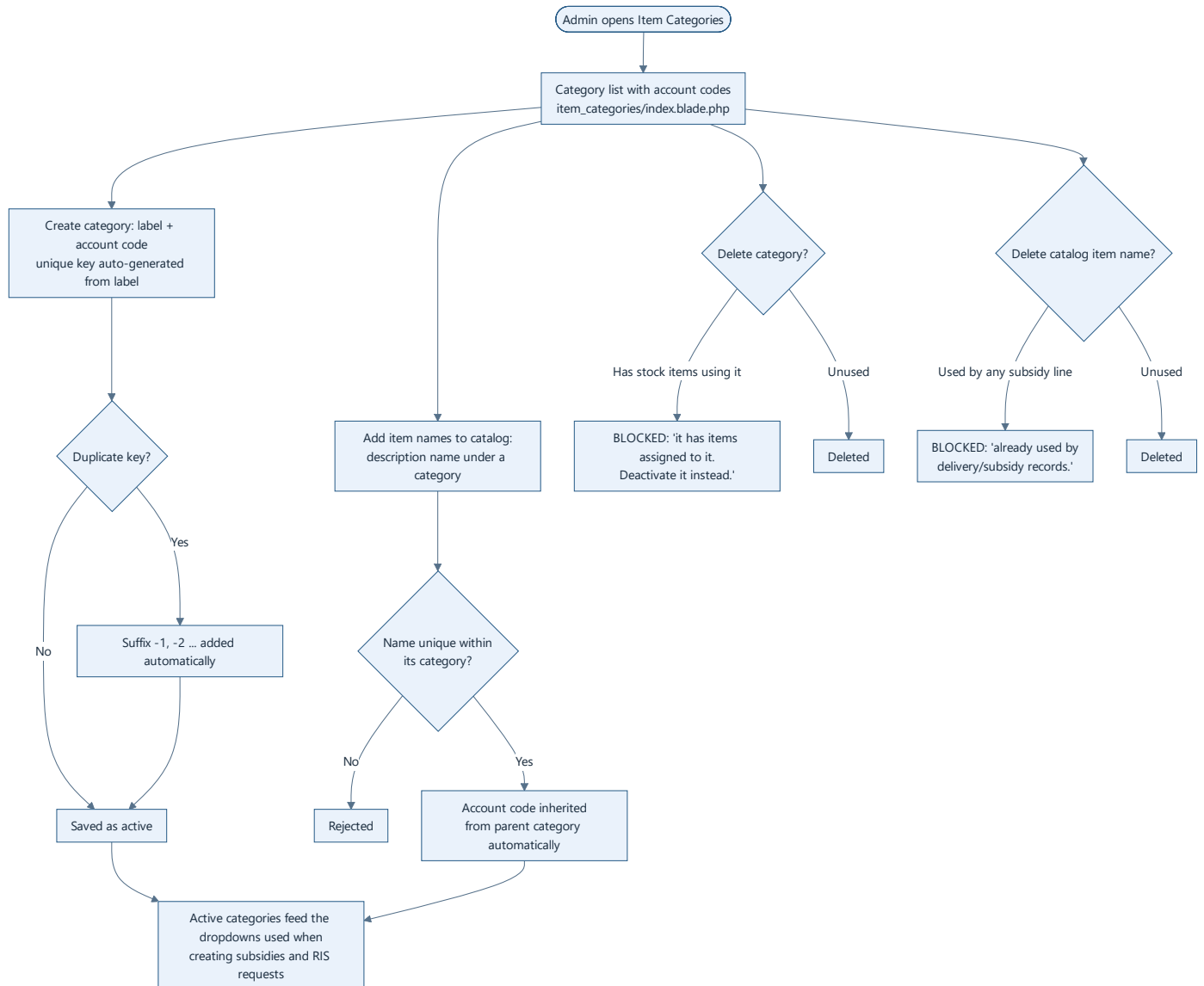
- Login rate limit: **10 attempts / minute** per username+IP [AuthController.php:28-33]
- Inactive accounts fail login exactly like wrong passwords and are also **logged out mid-session** if deactivated later [EnsureUserIsActive.php:20-31]
- Session ID regenerated at login; CSRF token regenerated at logout [AuthController.php:48-50, 82-83]
- All pages sent with `no-store` cache headers → browser back-button cannot redisplay protected pages [NoCache.php:29-51]

2.4 Files behind each step

Flow step	File(s)
Routes <code>/login</code> , <code>/logout</code> , dashboard	<code>routes/web.php:24-30</code>
Login logic, rate limiting, redirects	<code>app/Http/Controllers/AuthController.php</code> (login: lines 18-77, logout: 79-91)
Role checks on every request	<code>app/Http/Middleware/AdminOnly.php</code> (<code>admin</code>), <code>AdminWriteOnly.php</code> (<code>admin.write</code>), <code>AdminCreateOnly.php</code> (<code>admin.create</code>), <code>AdminOnlyStrict.php</code> (<code>admin.only.strict</code>), <code>EnsureUserIsActive.php</code> , <code>NoCache.php</code>
Middleware registration	<code>bootstrap/app.php:14-28</code>
Role helper methods	<code>app/Models/User.php</code> (<code>isAdmin</code> , <code>isWarehouseManager</code> , <code>hasAdminAccess</code> , <code>canWrite</code> , <code>canCreate</code> , <code>canApprove</code> , <code>isWarehouseUser</code>)
Per-role dashboards	<code>app/Http/Controllers/DashboardController.php</code> + views <code>resources/views/dashboard/admin.blade.php</code> , <code>dashboard/warehouse.blade.php</code> , <code>dashboard/no_warehouse.blade.php</code>
Warehouse scoping of all queries	<code>app/Http/Controllers/Concerns/ScopesWarehouse.php</code>

PART 3 — ITEM & ITEM CATEGORY MANAGEMENT (Master Data)

3.1 Flowchart



3.2 How stock records (Items) relate to the catalog

The catalog (category + item name + account code) is only the *vocabulary*. Actual inventory rows are created automatically when a delivery is recorded (see Part 4). One catalog item can therefore have **many** stock records — one per warehouse, per unit cost, per expiry date, per originating subsidy.

Display rule: identical-looking records (same description + unit cost + ENGAS cost + expiry + warehouse) are grouped in the Items list with a **“Merged (N)”** badge showing summed quantity [ItemController.php:92-114].

3.3 Files behind each step

Flow step	File(s)
Category CRUD (admin only)	<code>app/Http/Controllers/ItemCategoryController.php</code> (store: 23–61, destroy guard: 83–96)
Catalog item CRUD (admin only)	<code>app/Http/Controllers/ItemCatalogItemController.php</code> (delete guard: 61–72)
Stock record list / merged badge / create / delete guards	<code>app/Http/Controllers/ItemController.php</code> (index grouping: 92–114, manual create: 135–199, delete guards: 260–294)
Models	<code>app/Models/ItemCategory.php</code> (60s-cached active list), <code>app/Models/ItemCatalogItem.php</code> , <code>app/Models/Item.php</code>
Views	<code>resources/views/item_categories/index.blade.php</code> , <code>resources/views/items/*.blade.php</code>

PART 4 — SUPPLIER / SUBSIDY / DELIVERY FLOW (detailed)

This is the **inbound** pipeline: goods arriving from a supplier into warehouses.

4.1 Stage 1 — Create the Subsidy (*Admin or Warehouse Manager*)

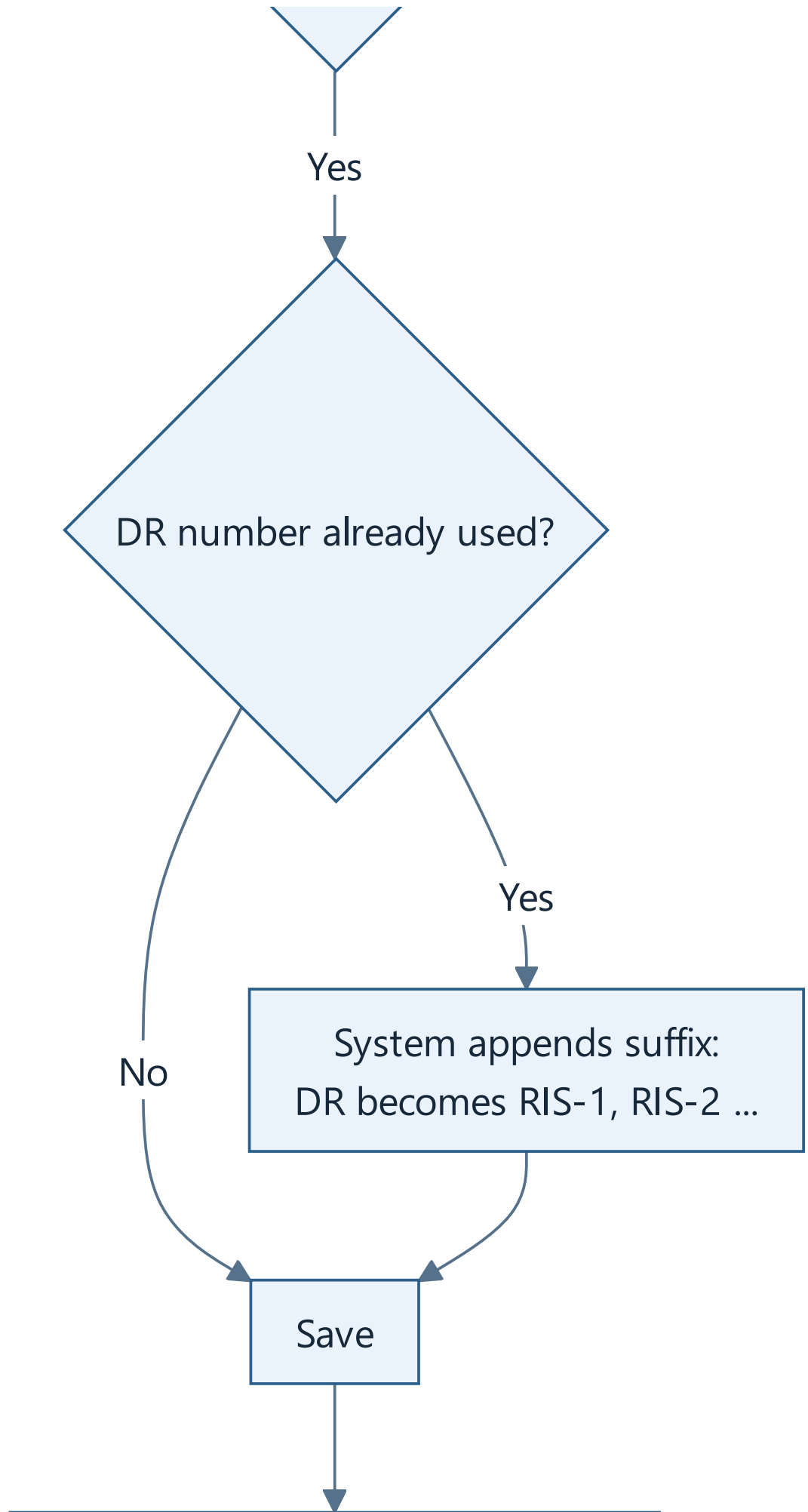
Subsidies list > New Subsidy

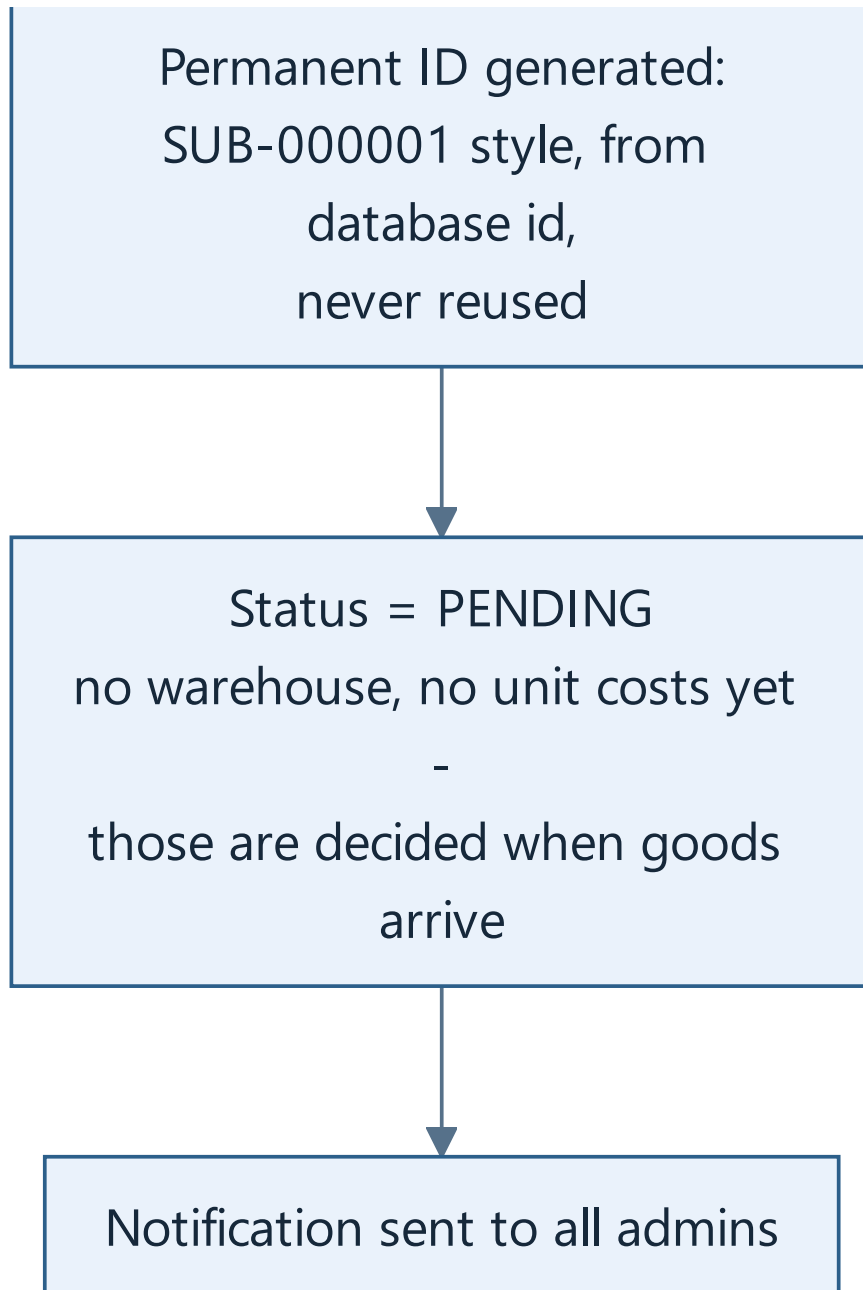
Enter RIS information:
RIS number, supplier, delivery
date,
place of delivery, remarks

Add requested item lines:
description from catalog, unit,
category,
quantity requested, optional
expiry date

At least 1 line and
quantities valid?

No

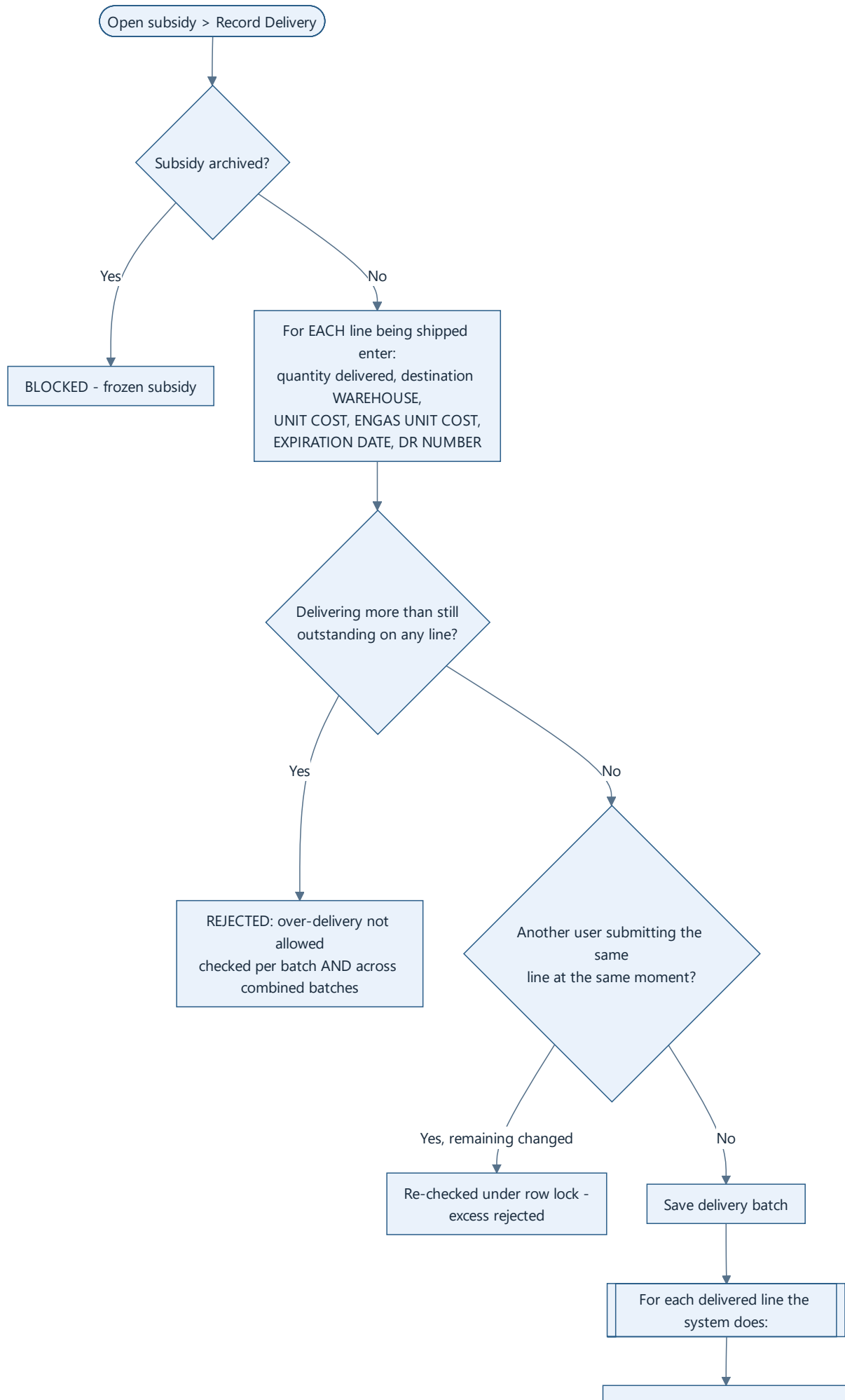


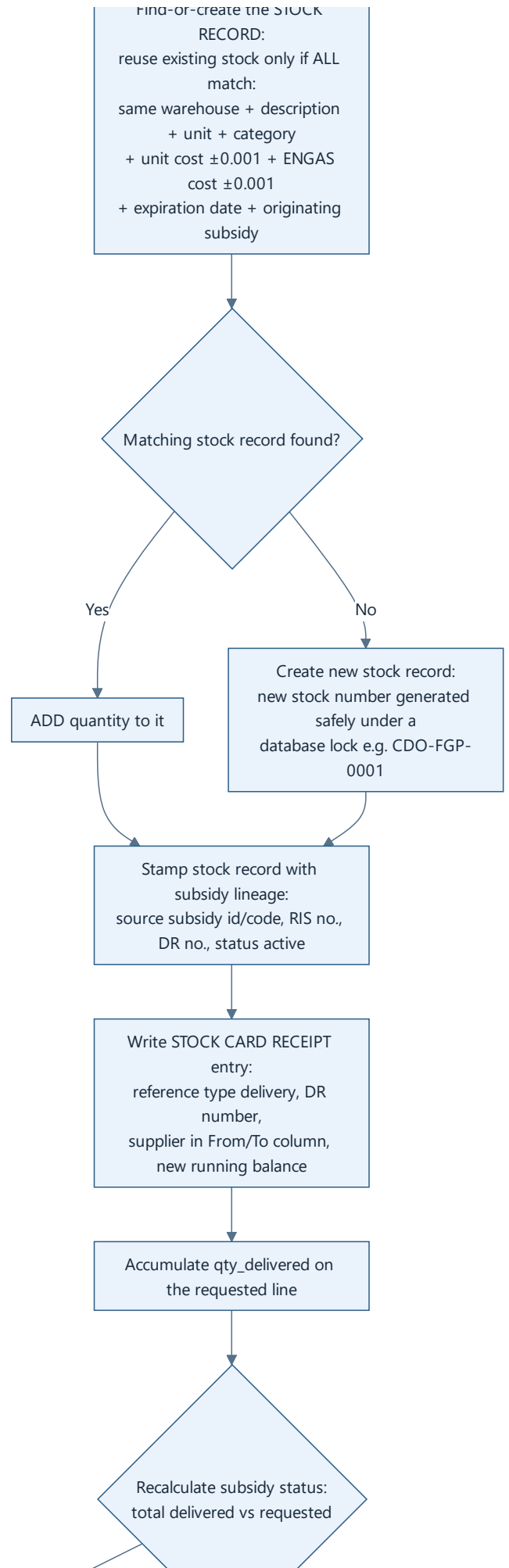


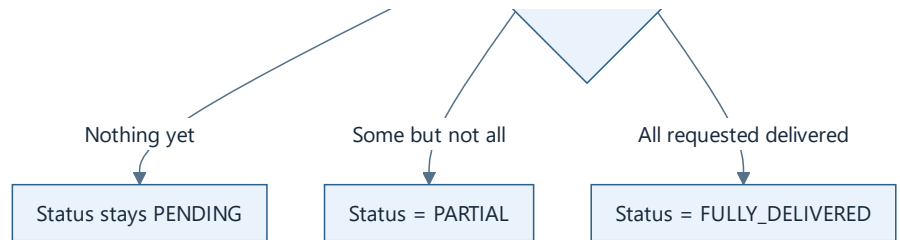
Important: `quantity_requested` is the original demand and is **never recalculated** later by delivered amounts. Delivered quantities are tracked separately beside it. `[routes note: DeliverySubsidyController::store lines 143-236]`

4.2 Stage 2 — Record a Delivery / Dispatch batch (*Admin or Warehouse Manager*)

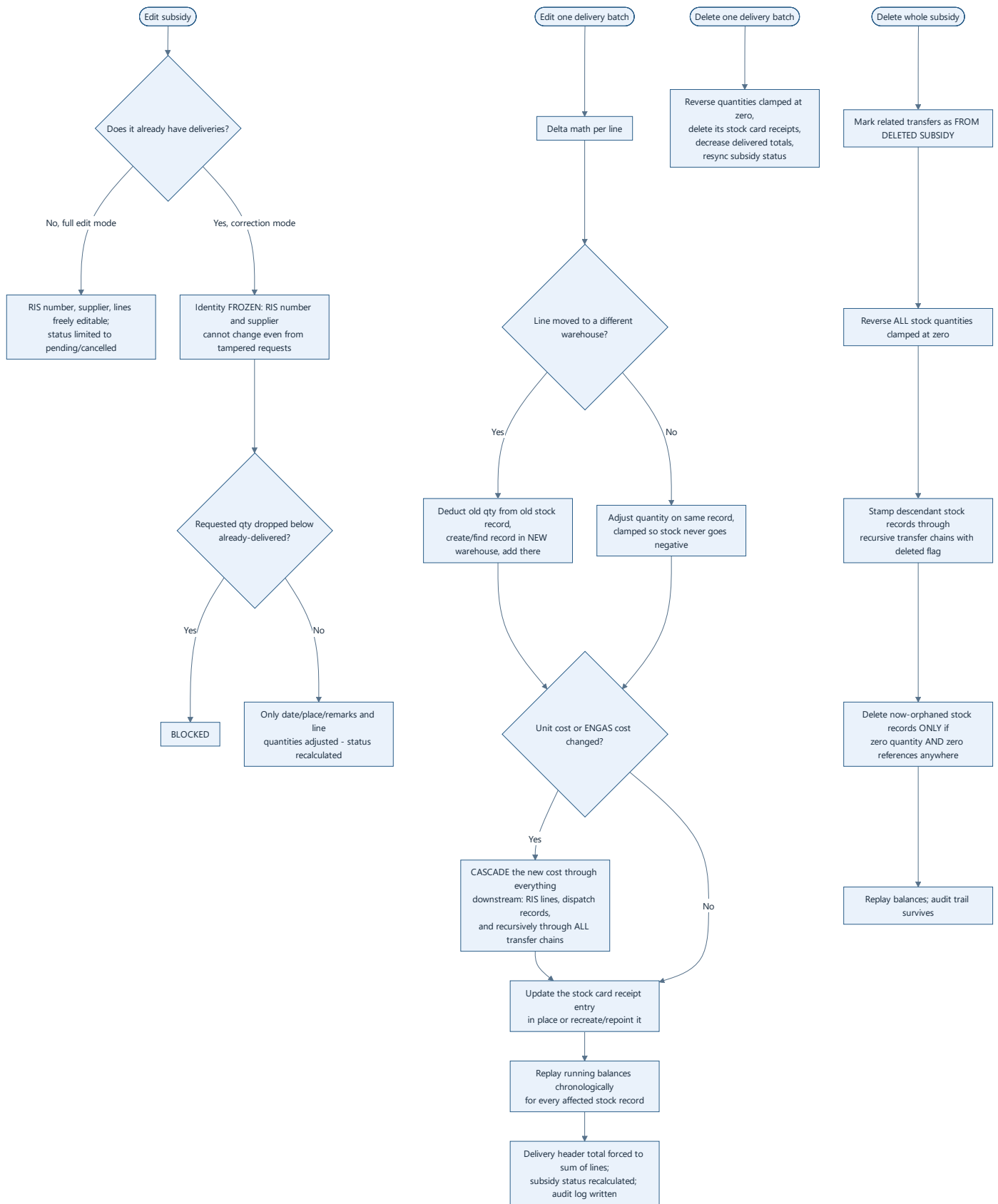
One subsidy can be fulfilled by **multiple partial deliveries**.







4.3 Stage 3 — Corrections and reversals (*Admin only*)



Archive/Restore: archiving freezes a subsidy (**blocks new deliveries**) and flags its transfers/items "FROM ARCHIVED SUBSIDY"; restore clears the flags. Nothing is ever silently lost — lineage snapshots keep traceability even after deletion or archiving.

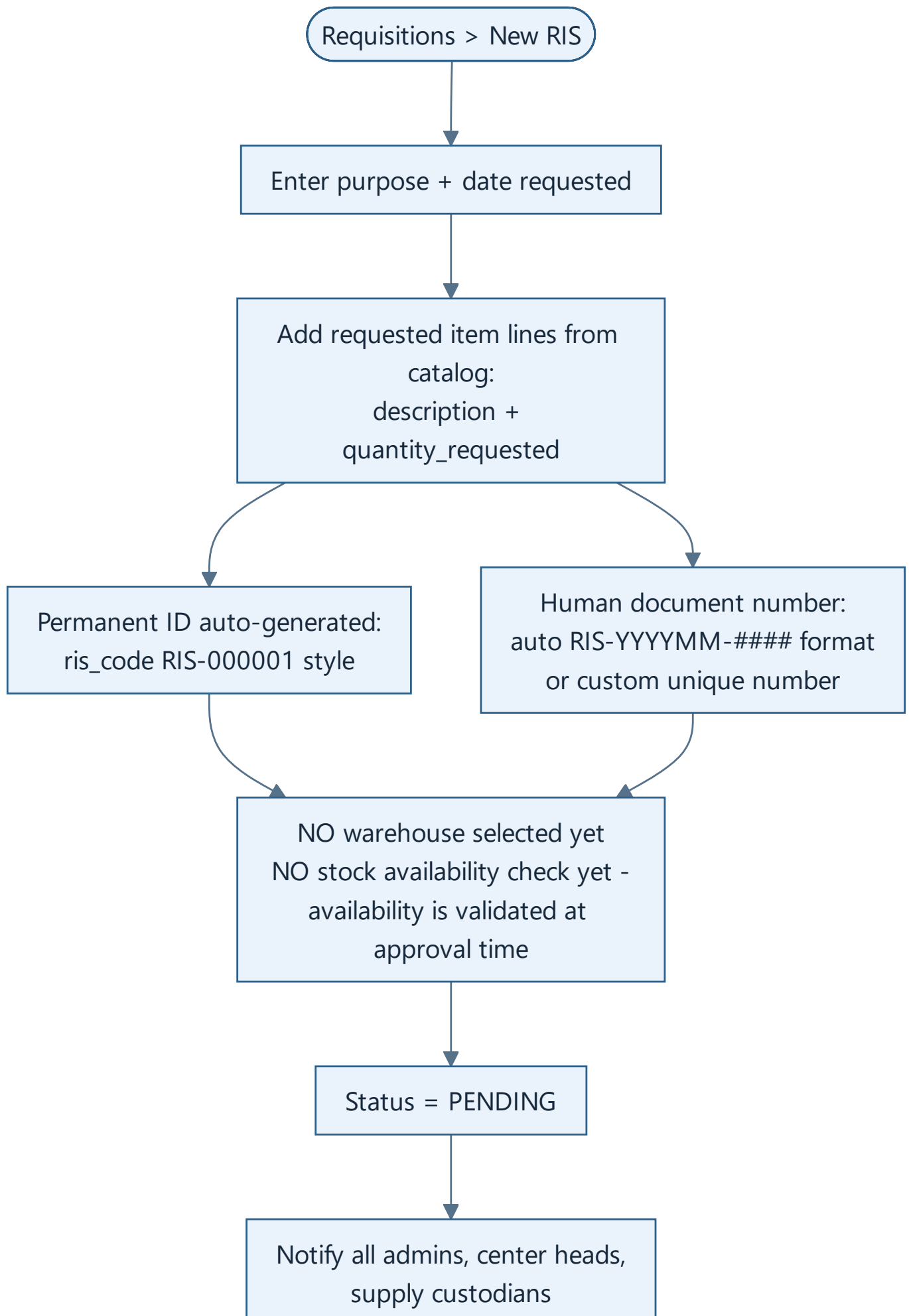
4.4 Files behind each step

Flow step	File(s)
Routes	<code>routes/web.php:62-87</code>
Create subsidy	<code>DeliverySubsidyController::store</code> (lines 143–236); <code>SUB-</code> code in <code>app/Models/DeliverySubsidy.php:23-29</code> ; DR duplicate check API <code>routes/web.php:216-218</code>
Status recalculation pending/partial/fully_delivered	<code>app/Models/DeliverySubsidy.php:updateDeliveryStatus</code> (lines 100–117)
Record delivery batch	<code>DeliverySubsidyController::storeDelivery</code> (lines 1113–1418)
Find-or-create stock record (identity rules)	<code>app/Models/Item.php:findOrCreateByUnitCost</code> (lines 265–365)
Safe stock-number generation	<code>app/Models/Item.php:generateStockNumber</code> (lines 367–417, MySQL GET_LOCK)
Auto-deactivate empty stock	<code>app/Models/Item.php</code> saving hook (lines 41–54)
Subsidy lineage snapshots	<code>app/Models/Item.php:applySubsidySnapshot</code> (lines 197–214)
Edit subsidy (correction vs full edit)	<code>DeliverySubsidyController::update</code> (lines 329–628)
Edit delivery batch + cascade	<code>DeliverySubsidyController::updateDelivery</code> (lines 1457–1832); cascade engine <code>app/Services/DeliverySubsidyCascadeService.php</code>
Delete delivery / subsidy reversal	<code>destroyDelivery</code> (1842–1900), <code>destroy</code> (630–768)
Archive / restore	<code>archive</code> (836–873), <code>restore</code> (880–935)
Audit trail view	<code>auditLog</code> action + <code>resources/views/delivery_subsidies/audit_log.blade.php</code>
Views	<code>resources/views/delivery_subsidies/*.blade.php</code> (create, delivery, show, edit, edit_delivery, audit_log)

PART 5 — REQUISITIONS / RIS FLOW (detailed)

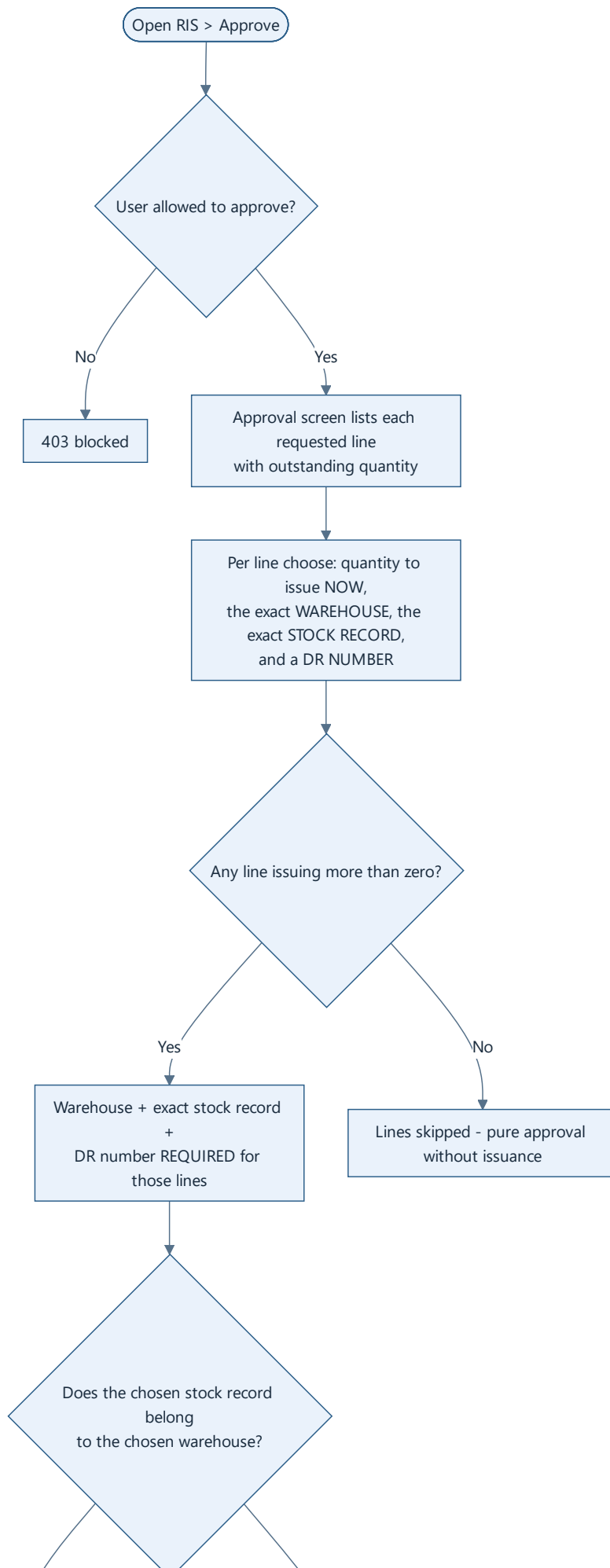
RIS = *Requisition and Issue Slip* (official DSWD form). This is the **outbound** pipeline: centers/offices requesting goods. (The menu label “Augmentations” refers to this same module — there is no separate augmentation entity.)

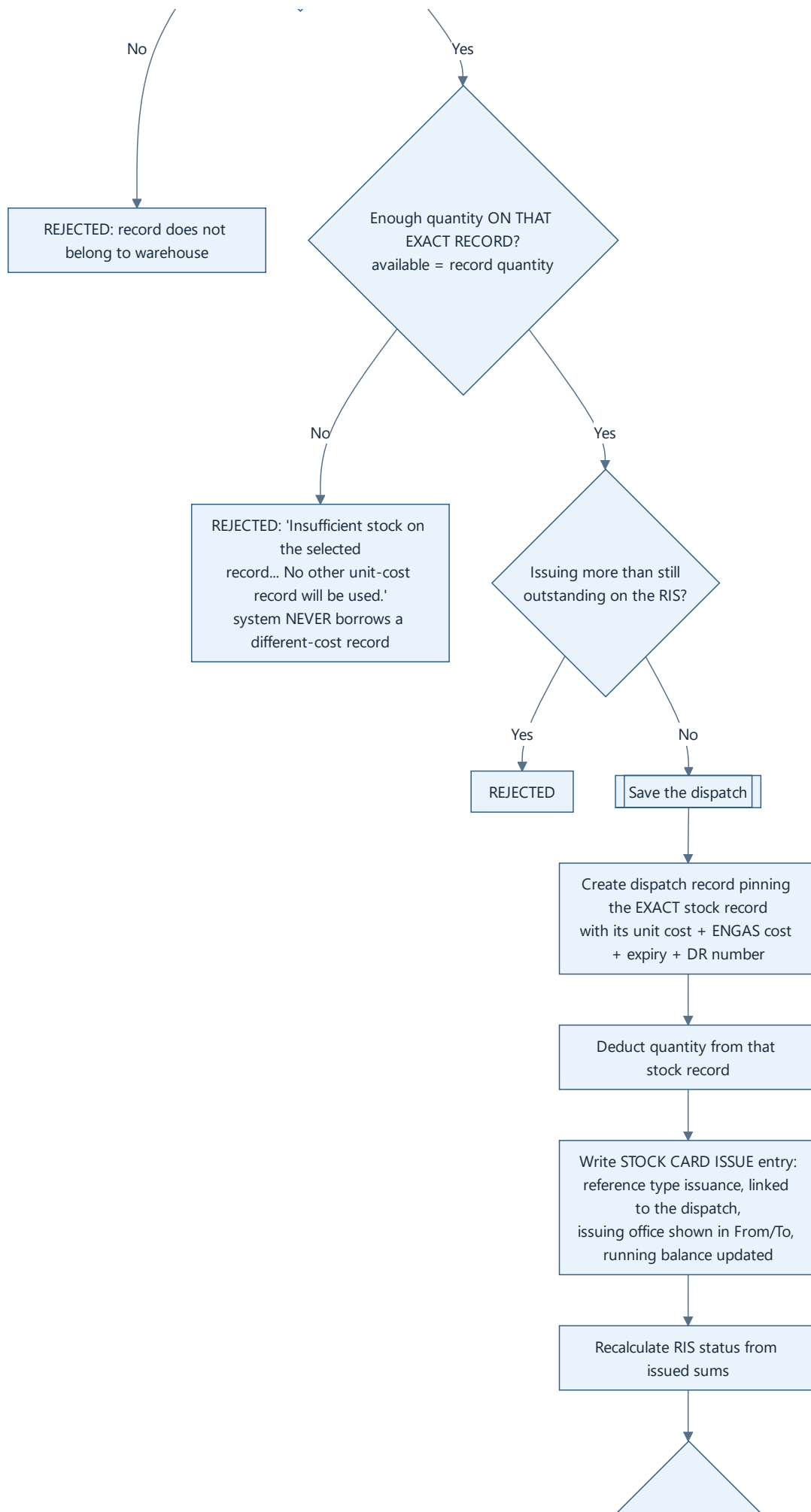
5.1 Stage 1 — Create the RIS (*Admin or Warehouse Manager*)

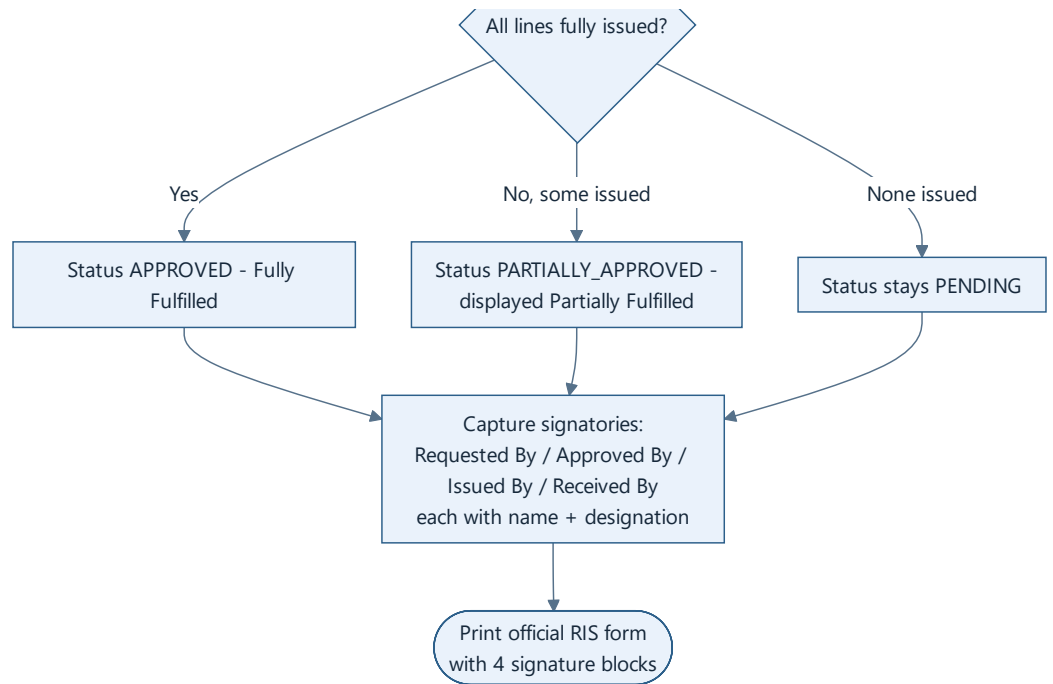


5.2 Stage 2 — Approve & Issue (*Admin, Warehouse Manager, Supply*)

Custodian, Center Head — NOT Center Staff)

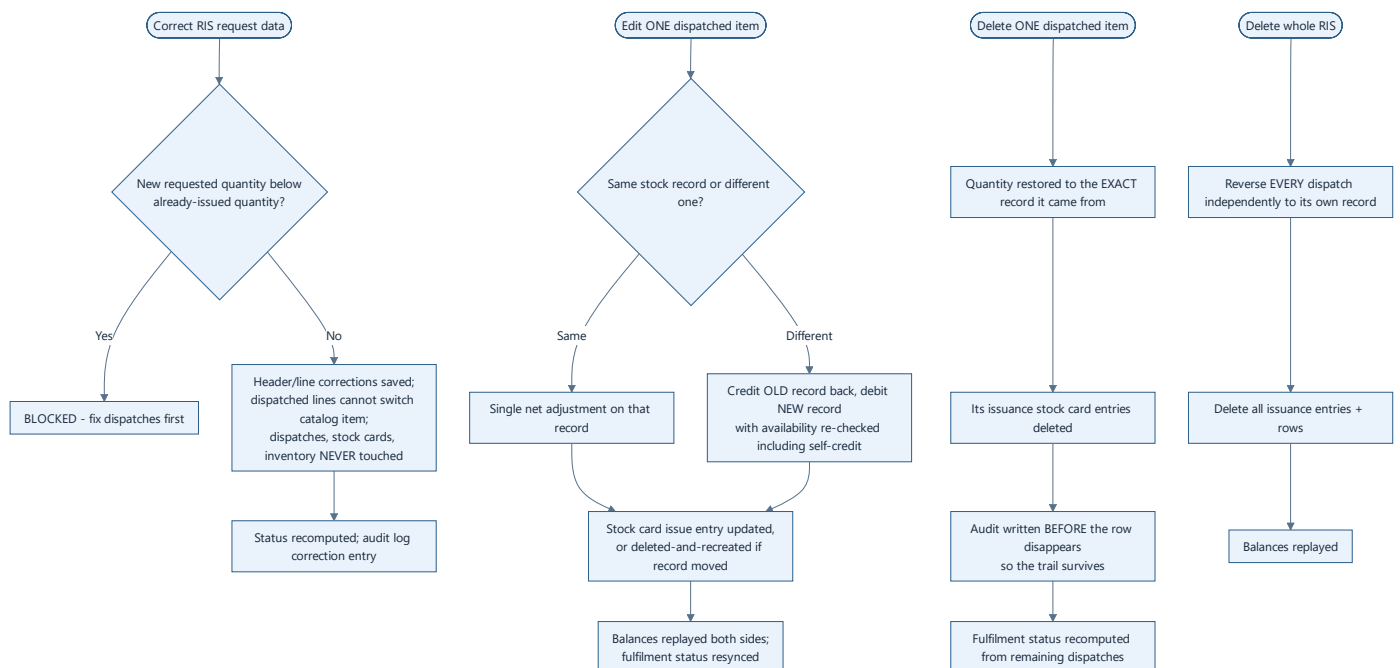






Partial fulfillment: repeat Stage 2 anytime — each run issues additional quantities from whatever exact records are chosen.

5.3 Stage 3 — Corrections and reversals *(Admin only)*



5.4 Files behind each step

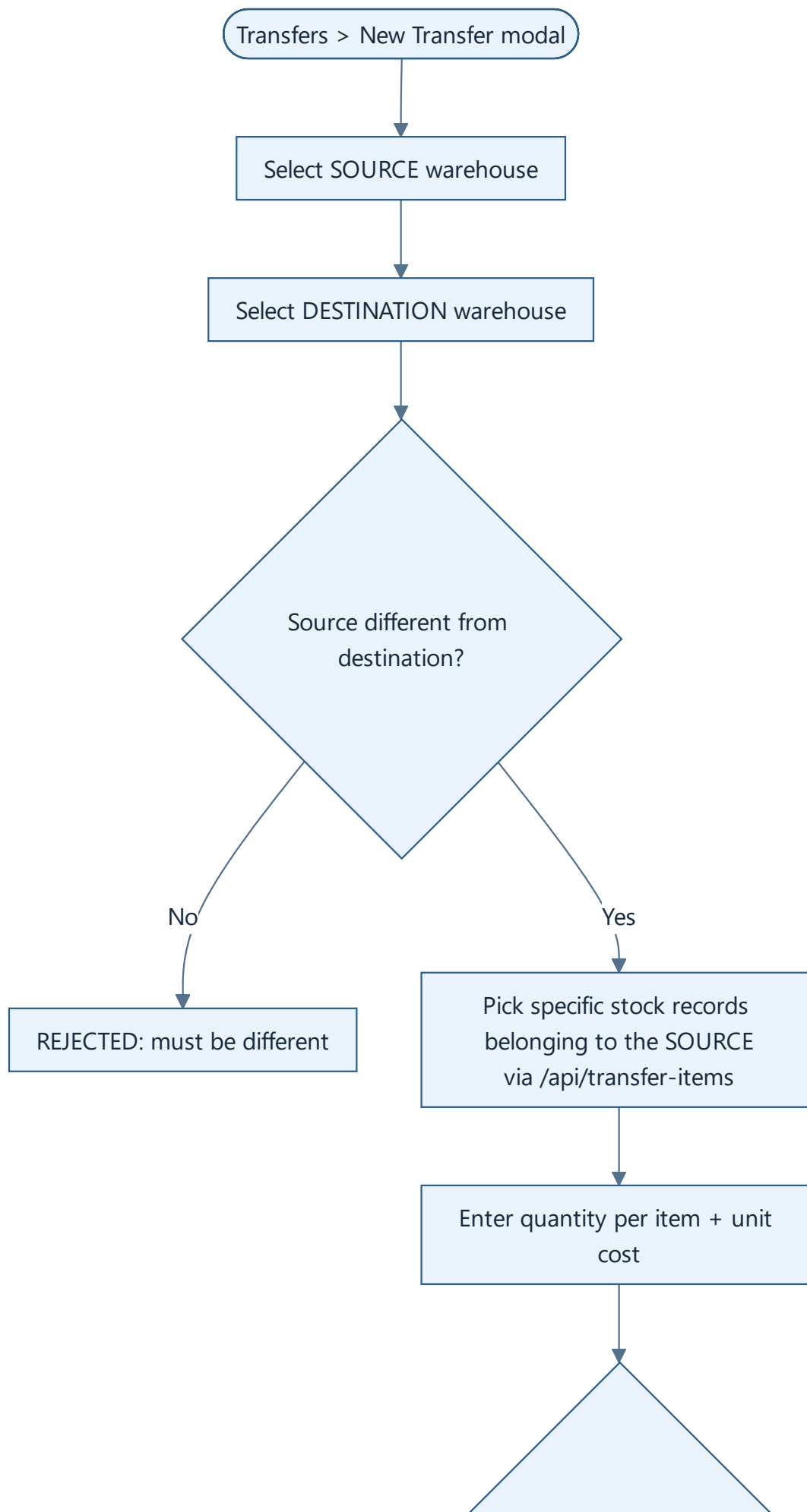
Flow step	File(s)
Routes	<code>routes/web.php:89-127</code>
Number generation	<code>ris_code</code> in <code>app/Models/Requisition.php:27-33</code> ; <code>ris_number</code> <code>RIS-YYYYMM-####</code> in <code>generateRisNumber</code> (245–262)
Create RIS	<code>RequisitionController::store</code> (204–329)

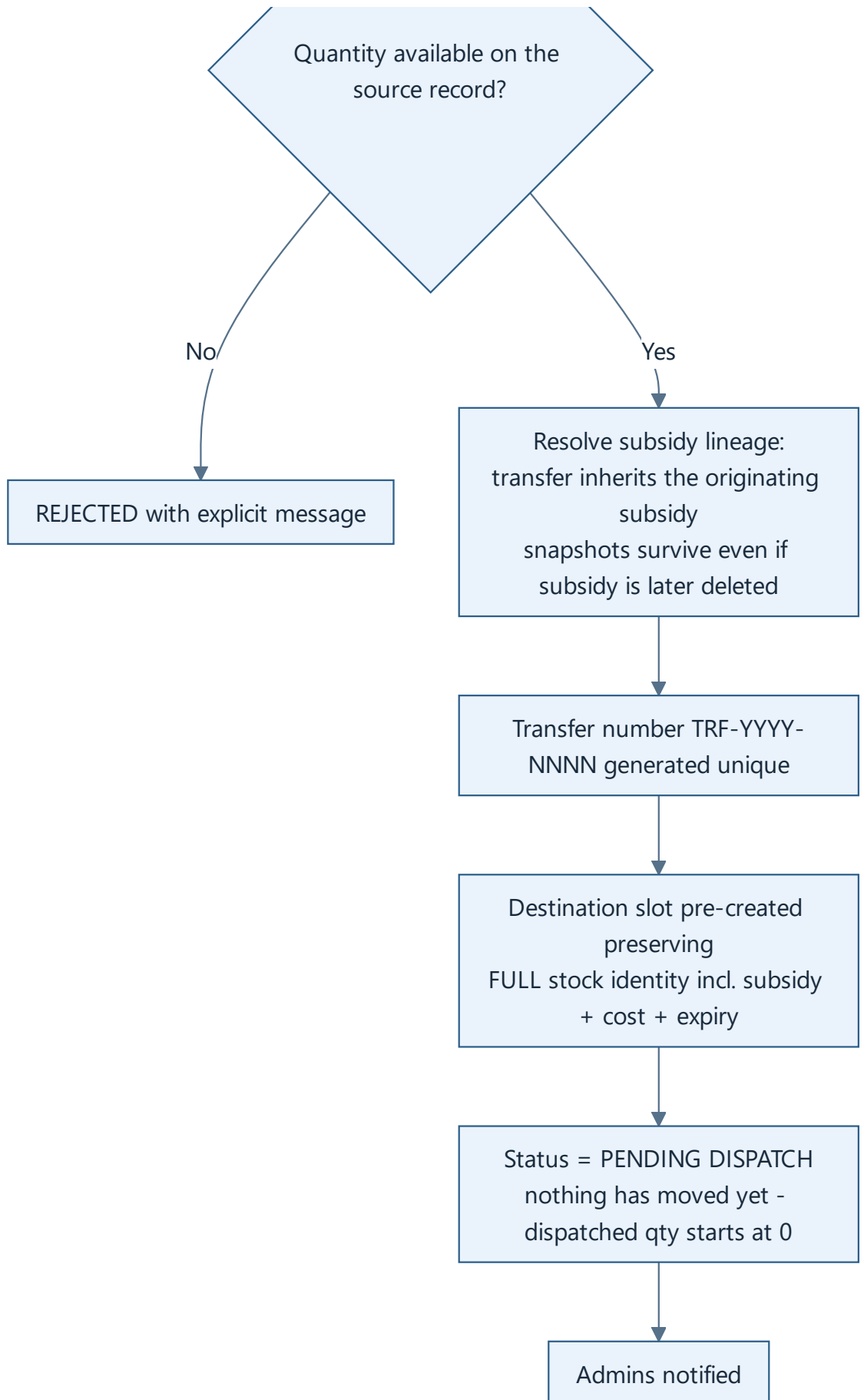
Flow step	File(s)
Approval gate (who may approve)	<code>User::canApprove</code> + controller lines 856–889
Issue/approve processing	<code>processApproval</code> (880–1073): exact-record lock 944–956, availability check 962–970, dispatch creation 985–994, stock card 998–1013, status 1027, signatories 1030–1039
Status recalculation	<code>app/Models/Requisition.php:updateFulfilmentStatus</code> (212–243)
Request-only correction	<code>correct()</code> (636–811)
Dispatch edit/delete	<code>updateDispatch</code> (1154–1341), <code>destroyDispatch</code> (1351–1437), whole-RIS <code>destroy</code> (511–567)
Print RIS	<code>printRis</code> (1461–1468) + <code>resources/views/requisitions/print.blade.php</code>
Helper APIs	<code>/api/requisition-items</code> (items in a warehouse), <code>/api/requisition-description-items</code> — <code>routes/web.php:125-127</code>
Audit	<code>app/Models/RequisitionAuditLog.php</code> ; view <code>requisitions/audit_log.blade.php</code>
Views	<code>resources/views/requisitions/*.blade.php</code>

PART 6 — STOCK TRANSFER FLOW (detailed)

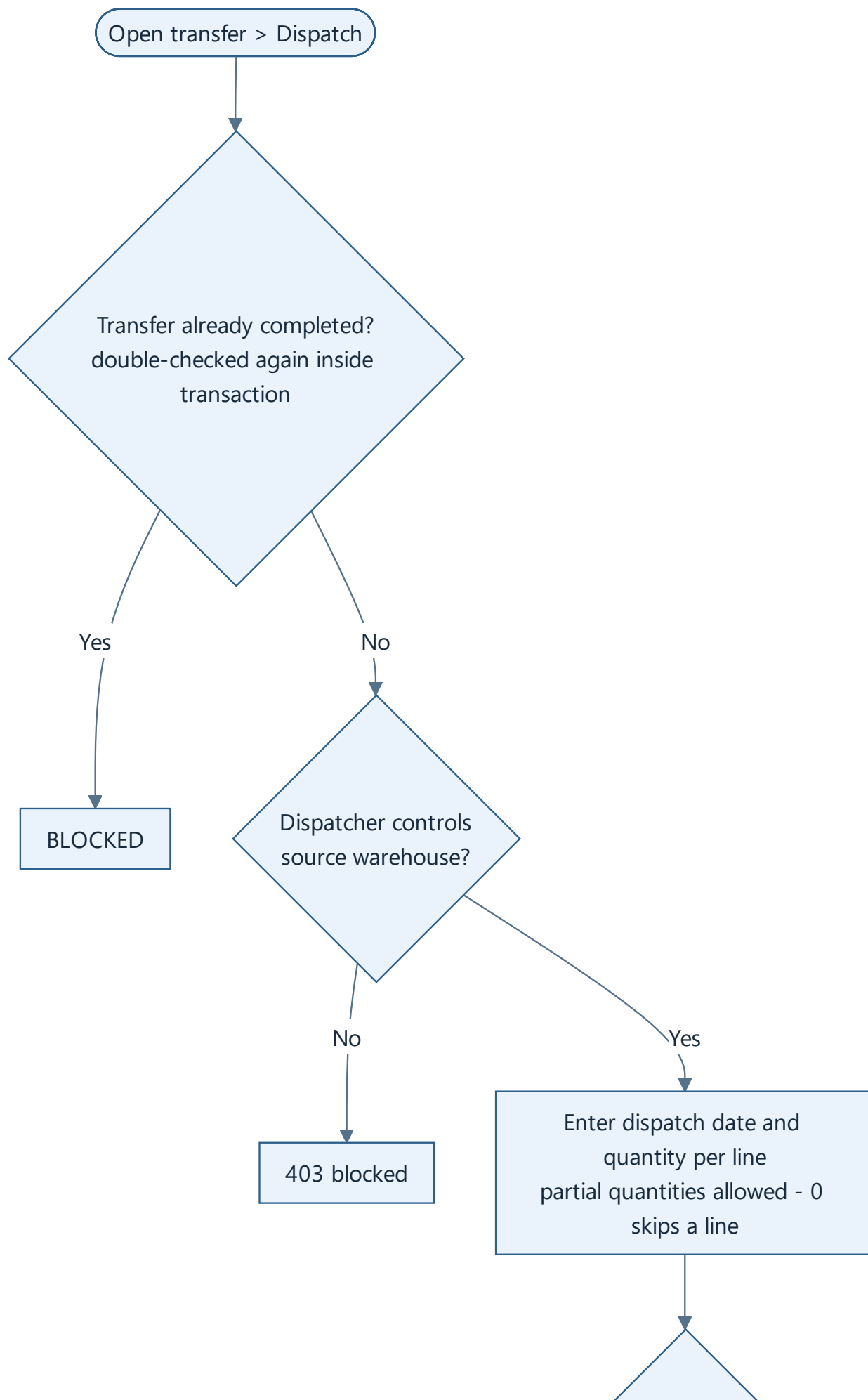
Moving stock between warehouses. Two steps by design: **Request** then **Dispatch** (partial dispatches allowed).

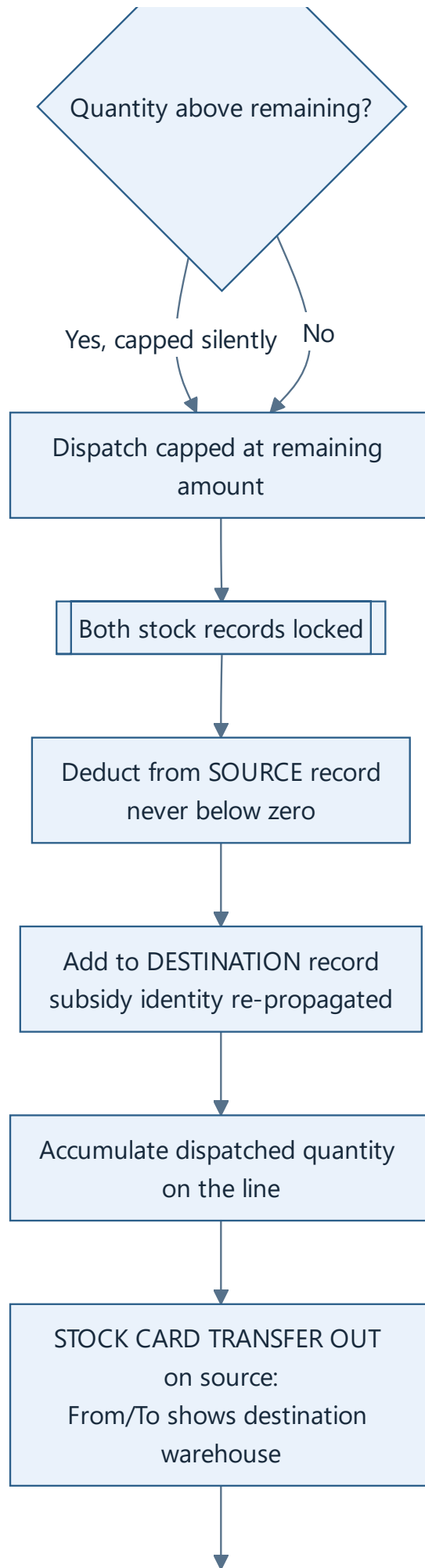
6.1 Stage 1 — Create transfer request (*Admin or Warehouse Manager*)

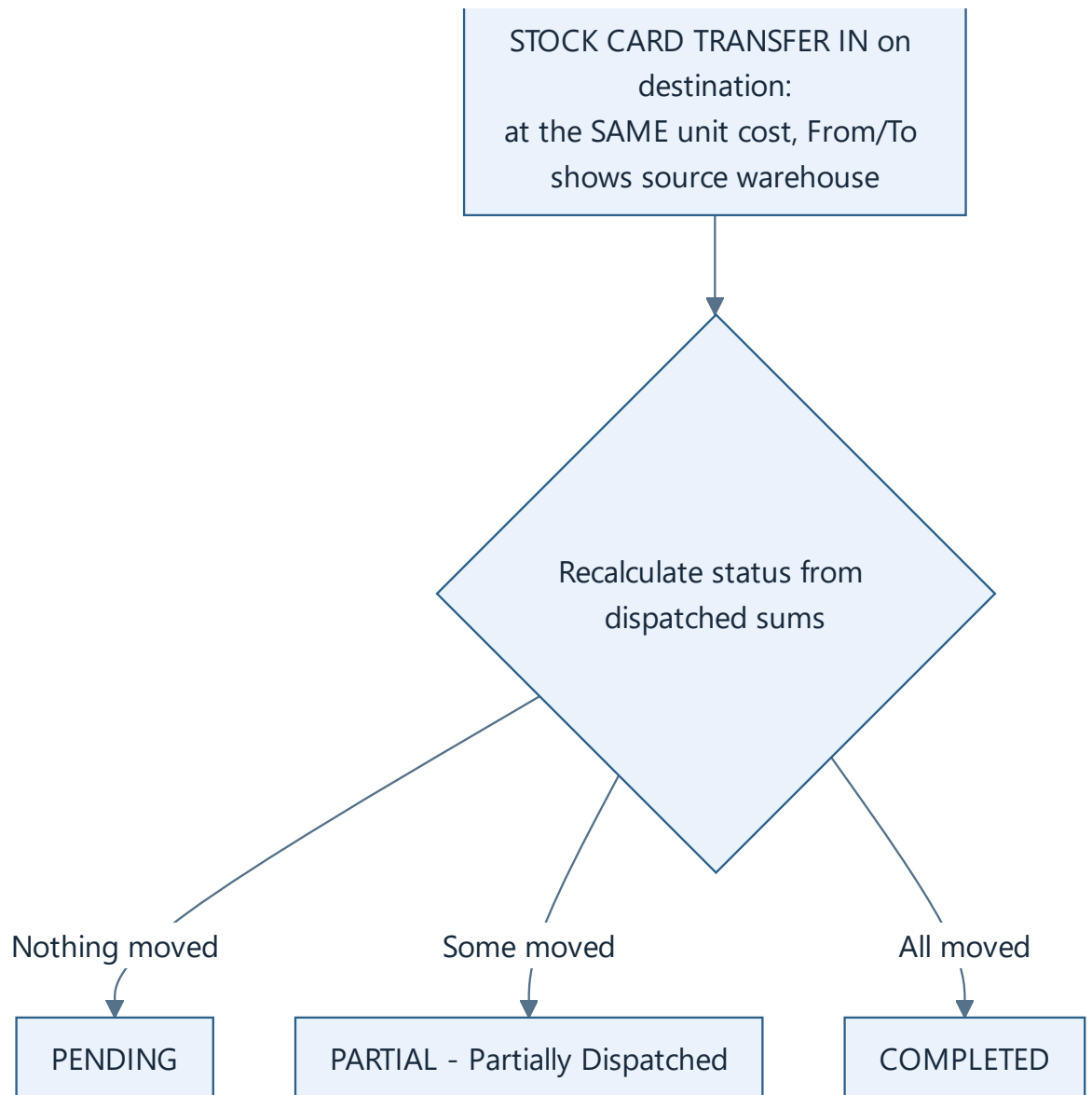




6.2 Stage 2 — Dispatch (*Admin or Warehouse Manager*)

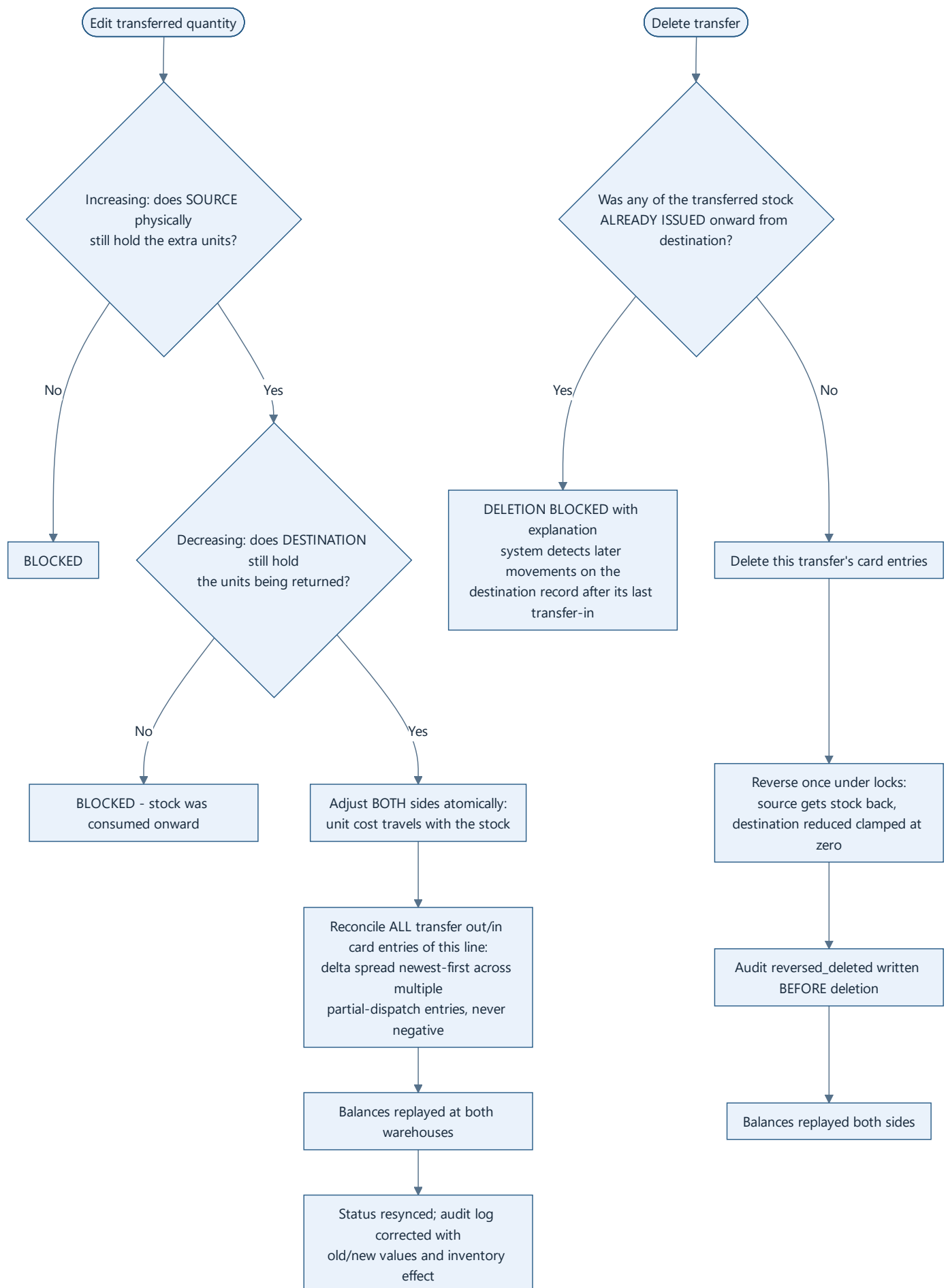






Multiple partial dispatches are supported — each creates another pair of transfer-out/transfer-in entries.

6.3 Stage 3 — Correction / Deletion (Admin only)



6.4 Files behind each step

Flow step	File(s)
Routes	<code>routes/web.php:138-154</code> ; item picker API <code>/api/transfer-items</code> line 142
Number generation TRF-YYYY-NNNN	<code>app/Models/StockTransfer.php:187-202</code>
Create request + lineage resolution + pre-created destination slot	<code>StockTransferController::store</code> (111–272)
Dispatch (both-side moves + dual card entries)	<code>dispatch</code> / <code>processDispatch</code> (277–440)
Status pending/partial/completed	<code>app/Models/StockTransfer.php:updateTransferStatus</code> (84–104)
Two-sided correction with newest-first card reconciliation	<code>update</code> (582–816; delta spread helper 828–853)
Deletion blocker + reversal	<code>destroy</code> (874–977), blocker check <code>destroyBlockers</code> (985–1022)
Audit	<code>app/Models/StockTransferAuditLog.php</code>
Views	<code>resources/views/transfers/*.blade.php</code> (index + create modal, dispatch, show, edit, print)

PART 7 — INVENTORY / STOCK MANAGEMENT & STOCK CARDS (detailed)

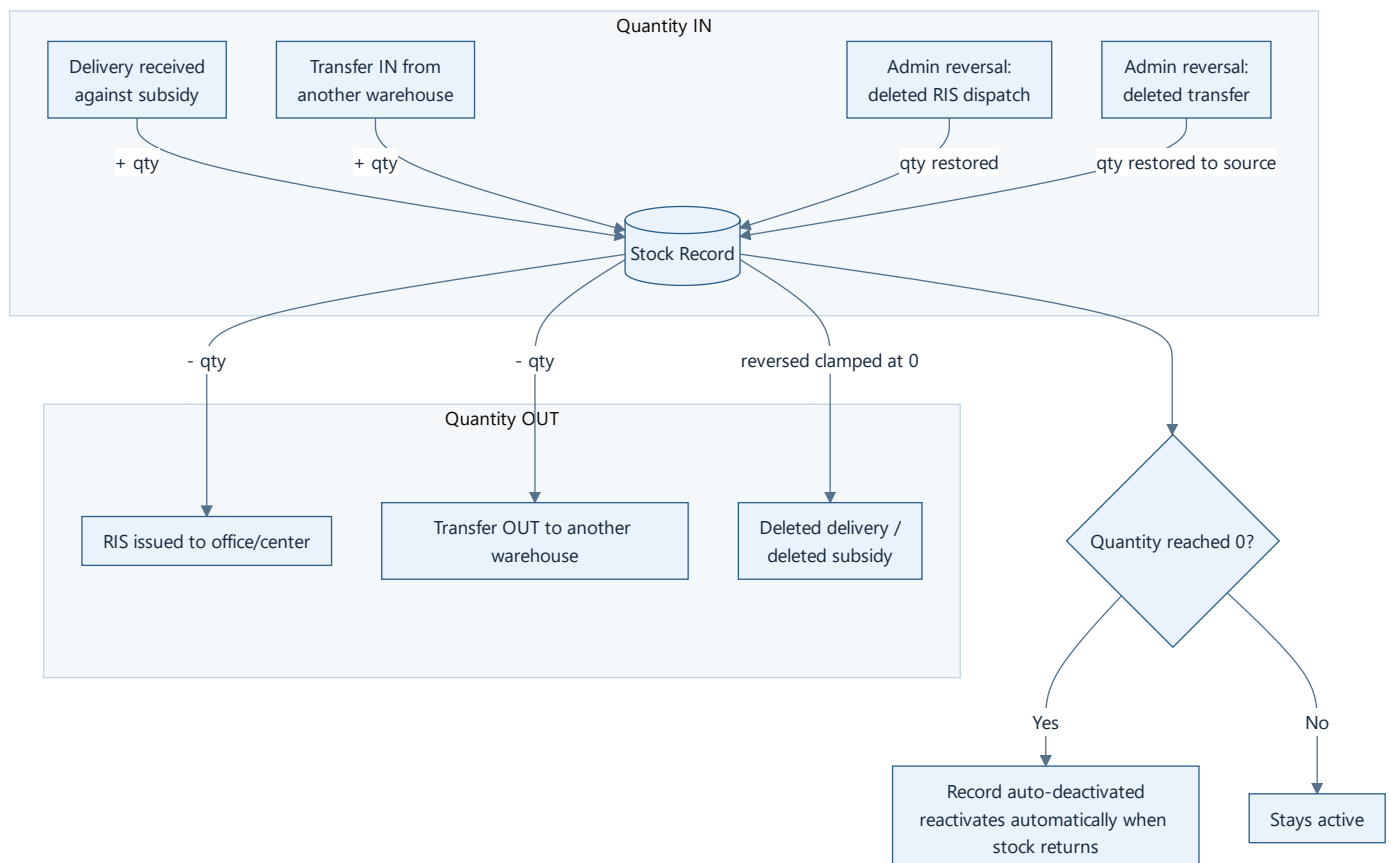
7.1 What a “stock record” is (the heart of the system)

Each row in the `items` table = one physical batch in one warehouse:

Field	Meaning
<code>stock_number</code>	Official card number, e.g. <code>CDO-FGP-0001</code> ({WAREHOUSE}-{CATEGORY}-####)
<code>quantity</code>	Current on-hand quantity
<code>unit_cost</code> / <code>engas_unit_cost</code>	Both travel together everywhere (reports show Engas Total Value = qty × ENGAS cost)
<code>expiration_date</code>	Optional; part of identity
<code>source_subsidy_*</code> columns	Lineage: which subsidy/RIS/DR this stock came from — survives transfers and subsidy deletion
<code>is_active</code>	Auto-deactivates when quantity reaches 0, reactivates when stock returns

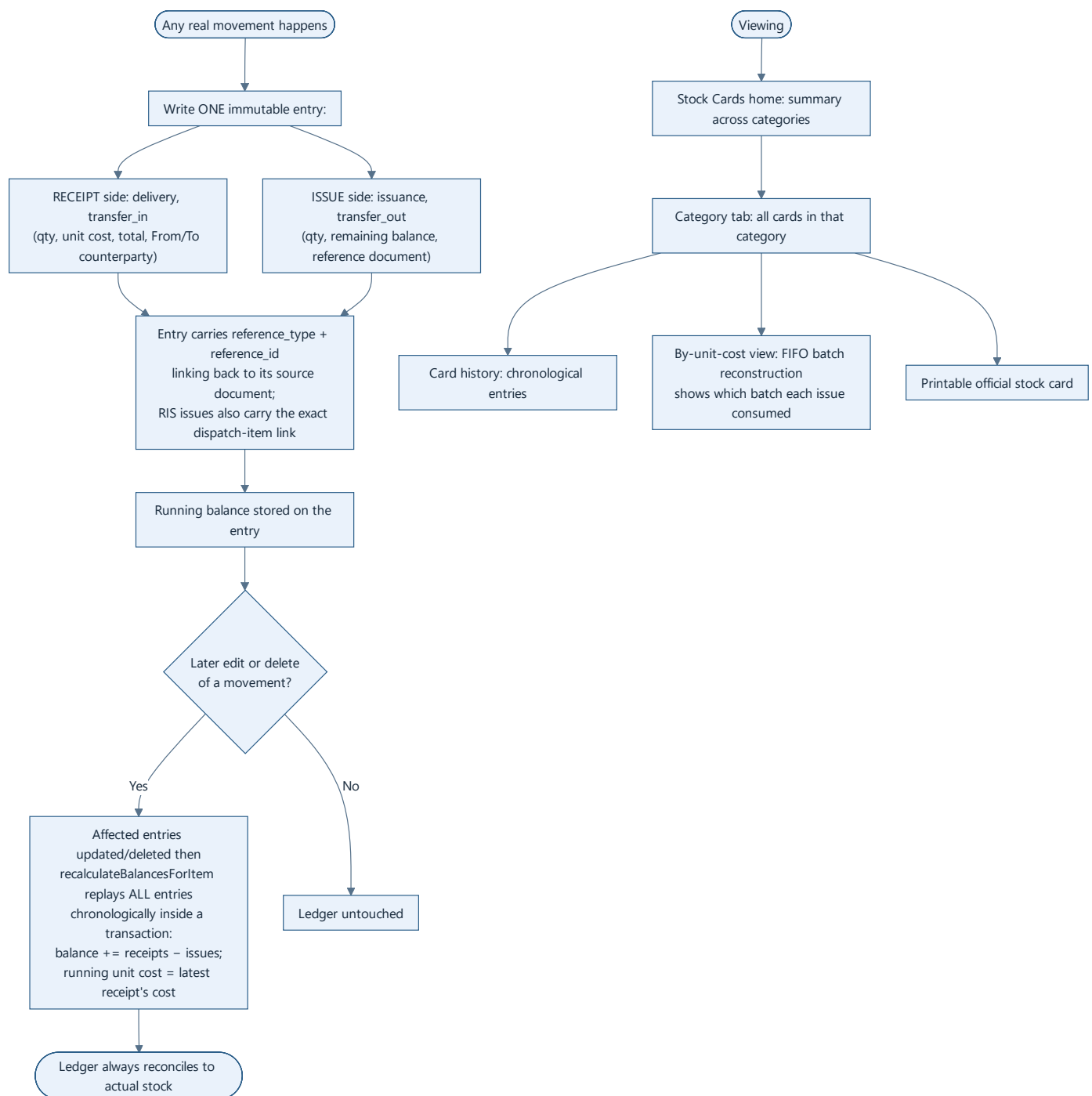
Identity rule: two records are “the same stock” only if warehouse + description + unit + category + unit cost + ENGAS cost + expiry + originating subsidy ALL match. Otherwise they stay separate batches. [`Item::findOrCreateByUnitCost`, `Item.php:265-365`]

7.2 Quantity movement map — every way stock changes



There are **NO** manual stock adjustments. Only real movements write to inventory.

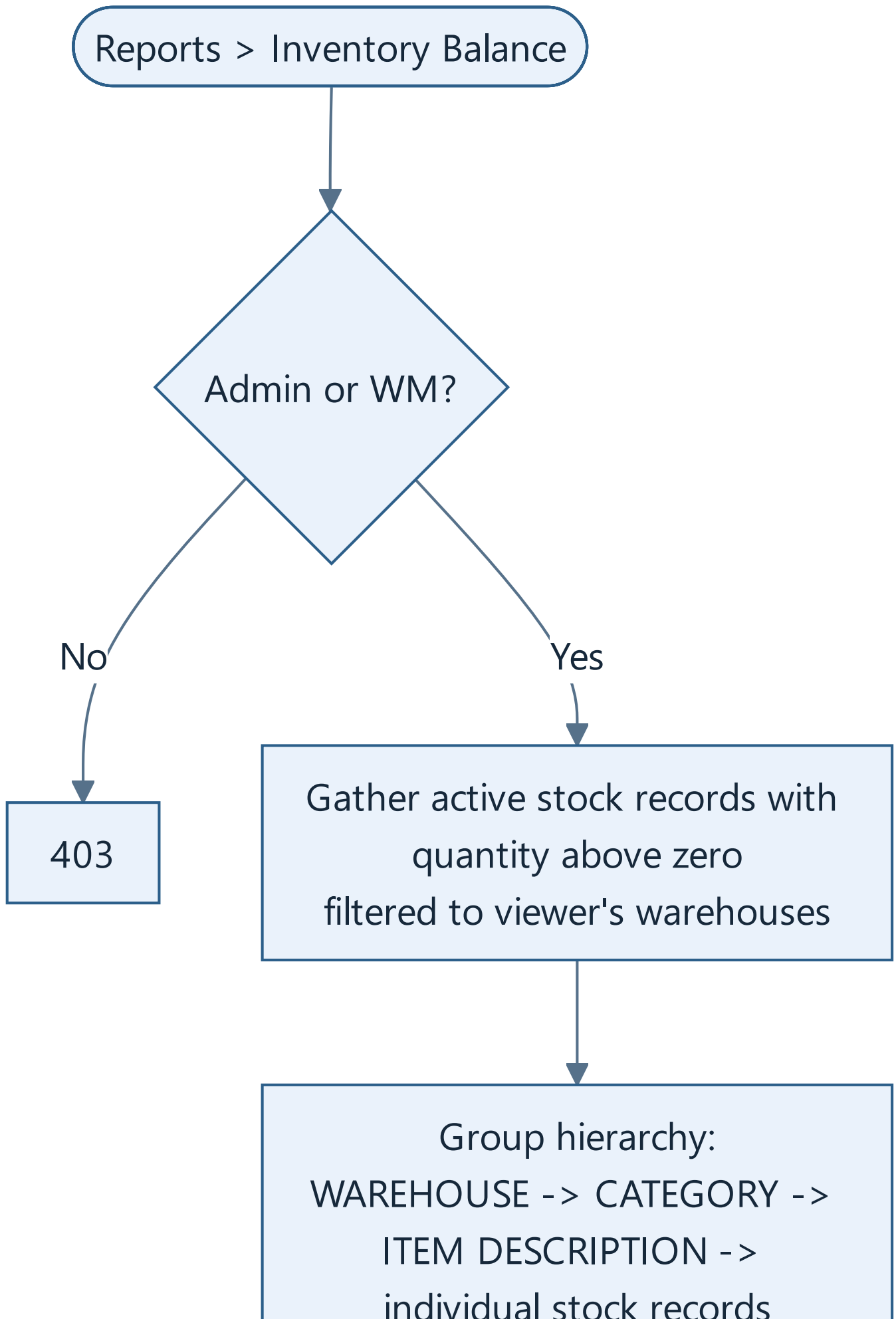
7.3 Stock Card engine (automatic ledger)

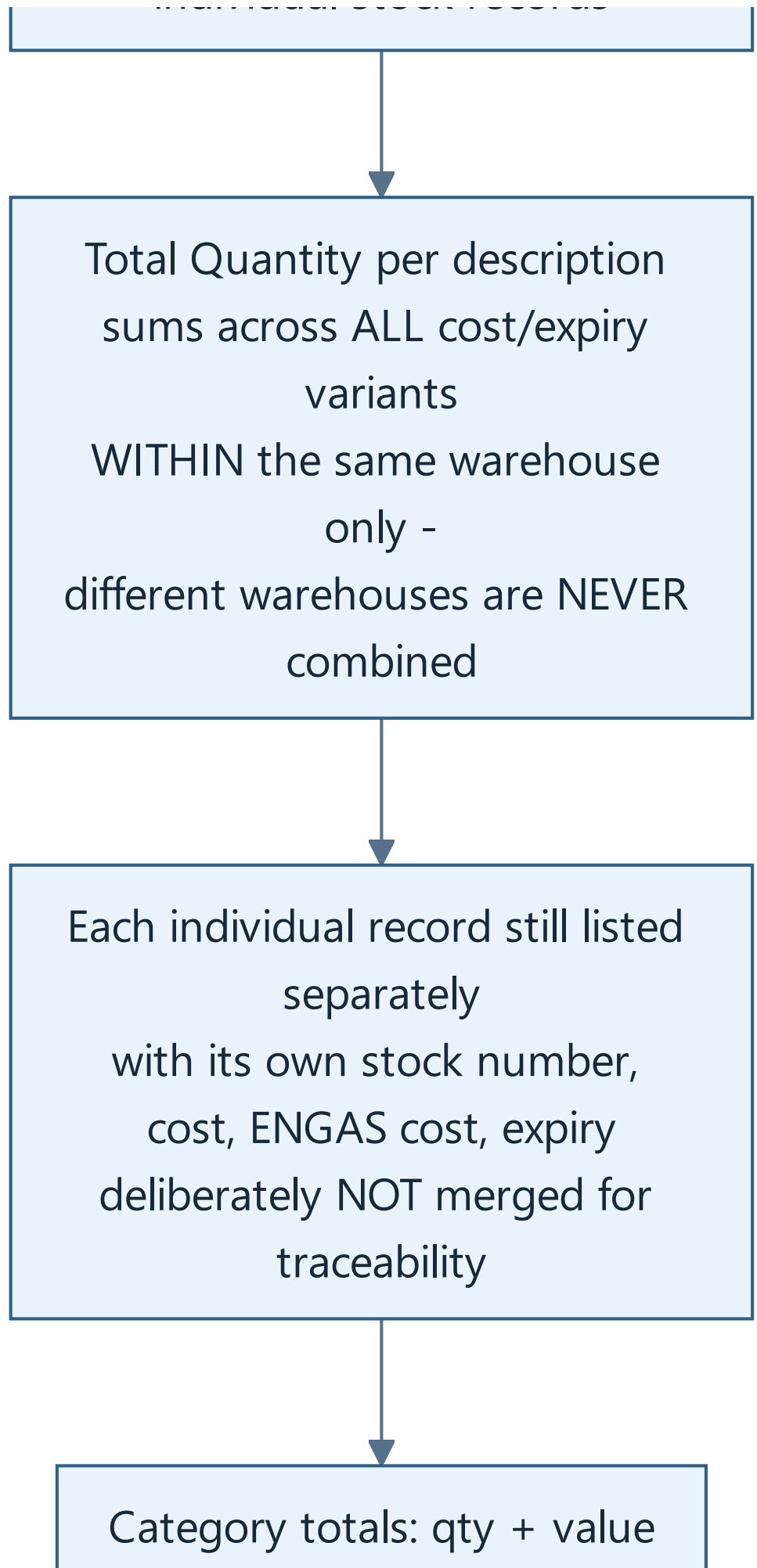


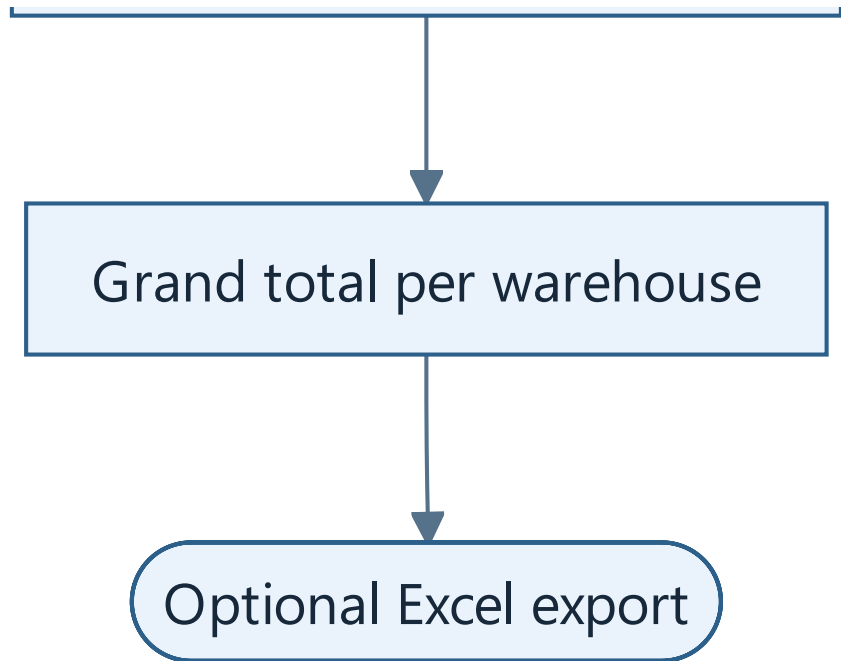
Warehouse protection: viewing a card requires access to that card's warehouse (403 otherwise)

[StockCardController.php:43-45, 61, 118].

7.4 Inventory Balance report (live) (*Admin + Warehouse Manager only*)





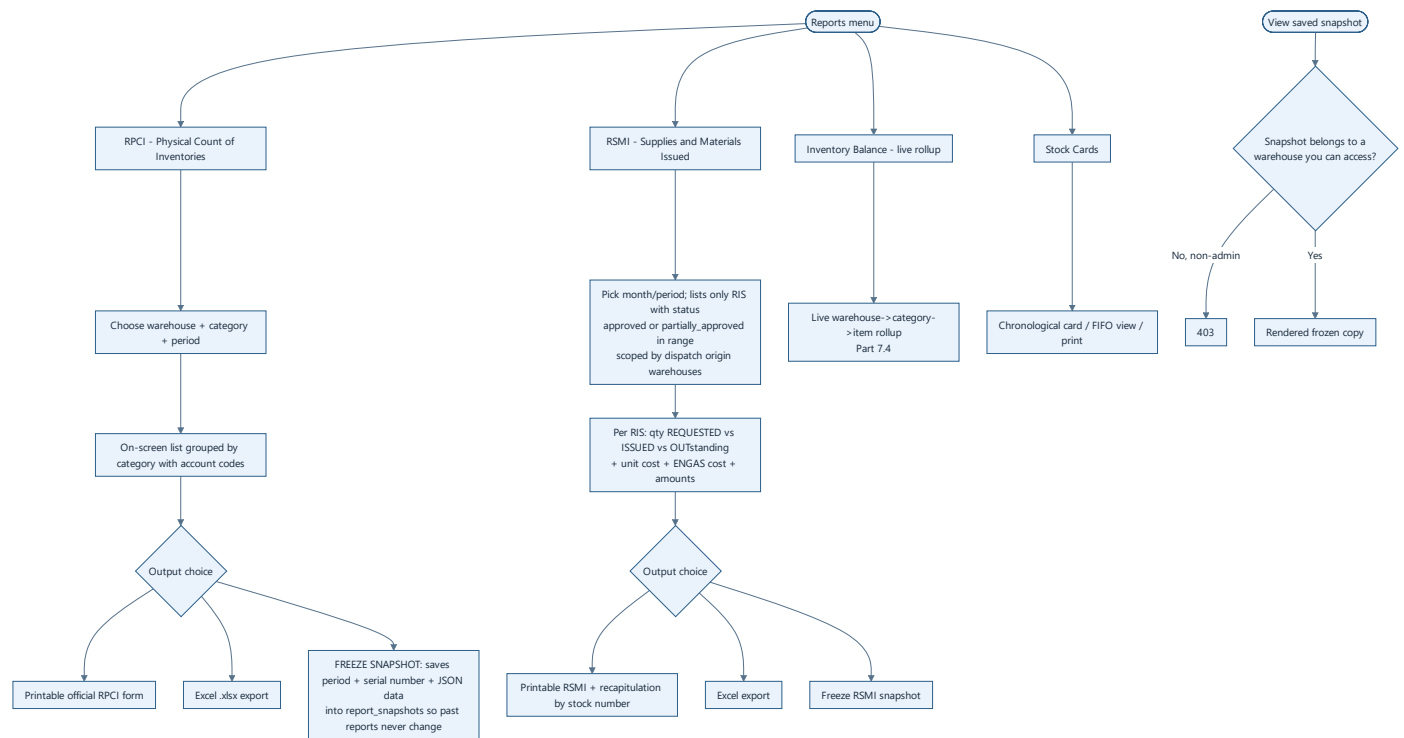


7.5 Files behind each step

Flow step	File(s)
Stock record identity/creation/deactivation	<code>app/Models/Item.php</code>
Stock card ledger model + balance replay	<code>app/Models/StockCardEntry.php:recalculateBalancesForItem</code> (38–65)
Who writes entries	<code>DeliverySubsidyController::storeDelivery</code> (delivery receipts), <code>RequisitionController::processApproval</code> (issuance), <code>StockTransferController::processDispatch</code> (transfer_in/out)
Stock card screens	<code>app/Http/Controllers/StockCardController.php</code> (summary 131–158, index 17–38, itemHistory 40–55, FIFO by-unit-cost 57–112, print 114–129) + <code>resources/views/stock_cards/*.blade.php</code>
Inventory Balance	<code>ReportController::inventoryBalance</code> (282–363) + export (580–688) + <code>resources/views/reports/inventory_balance.blade.php</code>

PART 8 — REPORTS

8.1 Report flows



8.2 Files behind each step

Report	Controller methods	Views
RPCI (list/print/export/snapshot)	<code>ReportController::rpci</code> (19–43), <code>printRpci</code> (45–64), <code>saveRpciSnapshot</code> (66–100), <code>exportRpci</code> (367–465)	<code>reports/rpci.blade.php</code> , <code>reports/rpci_print.blade.php</code> , <code>reports/snapshot.blade.php</code>
RSMI (list/print/export/snapshot)	<code>rsmi</code> (102–164), <code>printRsmi</code> (166–230), <code>saveRsmiSnapshot</code> (232–280), <code>exportRsmi</code> (467–578)	<code>reports/rsmi.blade.php</code> , <code>reports/rsmi_print.blade.php</code>
Inventory Balance (+export)	<code>inventoryBalance</code> (282–363), <code>exportInventoryBalance</code> (580–688)	<code>reports/inventory_balance.blade.php</code>
Snapshot viewing	<code>viewSnapshot</code> (690–705)	<code>reports/snapshot.blade.php</code>
Model	<code>app/Models/ReportSnapshot.php</code> (<code>report_type</code> <code>rpci/rsmi</code> , <code>period_month</code> , JSON payload)	

PART 9 — AUDIT & DATA INTEGRITY

9.1 Three append-only audit trails

Log table	Recorded actions	Written when
<code>delivery_subsidy_audit_logs</code>	<code>update</code> , <code>correction</code> , <code>archive</code> , <code>restore</code> (+cascade summaries)	Subsidy edited/corrected/archived/restored, delivery edited
<code>requisition_audit_logs</code>	<code>correction</code> , <code>dispatch_deleted</code>	RIS corrected; dispatched item deleted
<code>stock_transfer_audit_logs</code>	<code>corrected</code> , <code>reversed_deleted</code> , <code>subsidy_archived</code> , <code>subsidy_restored</code> , <code>subsidy_deleted</code>	Transfer corrections/deletions and subsidy state propagation

Design guarantee: audit rows about deletions are written **BEFORE** the parent row disappears, so the trail survives the deletion. Each row stores changed fields as JSON (transfers/subsidies also store the inventory effect).

9.2 Data-integrity rules the system enforces (QA checklist)

#	Rule	Where enforced
1	Requested quantities are NEVER overwritten by delivered/issued/transferred amounts	Separation throughout controllers; statuses derived from sums
2	Over-delivery of a subsidy line impossible (single batch and combined batches, tolerance $\epsilon=0.0001$)	<code>storeDelivery</code> 1201–1221; re-checked under row lock 1256–1262
3	RIS issuance only from the EXACT stock record; insufficient → reject, never borrow another cost record	<code>processApproval</code> 944–970
4	RIS availability validated at approval time, not at request time	Same block
5	Stock can never go negative — every deduction/reversal clamped ≥ 0	All reversal paths
6	Transfer source $\neq$ destination	Validation + DB-level check <code>store</code> L123–134
7	Transfer edits adjust BOTH warehouses atomically; increases require physical stock at source; decreases require units still held at destination	<code>StockTransferController::update</code> 615–671
8	Deleting a transfer blocked if its stock was already issued onward	<code>destroyBlockers</code> 985–1022
9	Every movement writes an immutable stock card entry; NO manual adjustments exist	Parts 4–7 pipelines
10	After any edit/delete, running balances are replayed chronologically inside a transaction	<code>StockCardEntry::recalculateBalancesForItem</code>
11	Stock keeps lineage (originating subsidy + snapshots) through transfers, subsidy deletion, and archiving (“FROM DELETED/ARCHIVED SUBSIDY” review badges)	<code>applySubsidySnapshot</code> + cascade service BFS walk
12	Deleting a subsidy hard-deletes orphan stock records only if zero quantity AND zero references	<code>destroy</code> 719–736
13	Categories/catalog items/suppliers can’t be deleted while referenced — deactivate instead	ItemCategoryController 83–96; ItemCatalogItemController 61–72; Supplier toggle only
14	Non-admin users see/mutate ONLY their assigned warehouses — server-side on every query incl. AJAX	<code>ScopesWarehouse</code> trait used by all controllers

#	Rule	Where enforced
15	Unauthorized direct URL access returns 403 (middleware + controller gates), never silent failure	Middleware chain (Part 2.2)
16	Cost changes cascade downstream through RIS lines, dispatches, and recursive transfer chains	<code>DeliverySubsidyCascadeService::cascadeItemCost</code>
17	Unit cost and ENGAS unit cost stay separate values everywhere (never collapsed)	Identity keys, dispatch lines, cards, reports
18	Concurrent submissions serialized with row locks (<code>lockForUpdate</code>) and advisory lock for stock numbers	storeDelivery, processApproval, processDispatch, generateStockNumber

APPENDIX — Quick file index by module

Module	Route definitions	Controller	Key models/services	Main views
Auth	<code>routes/web.php:24-26</code>	<code>AuthController</code>	<code>User</code> , middleware <code>EnsureUserIsActive</code> , <code>NoCache</code>	<code>auth/login.blade.php</code>
Dashboard	<code>routes/web.php:30</code>	<code>DashboardController</code>	—	<code>dashboard/{admin,warehouse,no_warehouse}.blade.php</code>
Item categories & catalog	<code>web.php:33-44</code>	<code>ItemCategoryController</code> , <code>ItemCatalogItemController</code>	<code>ItemCategory</code> , <code>ItemCatalogItem</code>	<code>item_categories/index.blade.php</code>
Items (stock records)	<code>web.php:47-59</code>	<code>ItemController</code>	<code>Item</code>	<code>items/*.blade.php</code>
Suppliers	<code>web.php:156-168</code>	<code>SupplierController</code>	<code>Supplier</code>	<code>suppliers/*.blade.php</code>
Warehouses	<code>web.php:183-196</code>	<code>WarehouseController</code>	<code>Warehouse</code>	<code>warehouses/*.blade.php</code>
Users	<code>web.php:198-211</code>	<code>UserController</code>	<code>User</code>	<code>users/*.blade.php</code>
Subsidies/Deliveries	<code>web.php:62-87</code>	<code>DeliverySubsidyController</code>	<code>DeliverySubsidy(+Item)</code> , <code>Delivery(+Item)</code> , <code>DeliverySubsidyAuditLog</code> , <code>DeliverySubsidyCascadeService</code>	<code>delivery_subsidies/*.blade.php</code>
Requisitions/RIS	<code>web.php:89-127</code>	<code>RequisitionController</code>	<code>Requisition(+Item,+DispatchItem)</code> , <code>RequisitionAuditLog</code>	<code>requisitions/*.blade.php</code>
Stock transfers	<code>web.php:138-154</code>	<code>StockTransferController</code>	<code>StockTransfer(+Item)</code> , <code>StockTransferAuditLog</code>	<code>transfers/*.blade.php</code>
Stock cards	<code>web.php:129-136</code>	<code>StockCardController</code>	<code>StockCardEntry</code>	<code>stock_cards/*.blade.php</code>
Reports	<code>web.php:170-181</code>	<code>ReportController</code>	<code>ReportSnapshot</code>	<code>reports/*.blade.php</code>
Notifications	<code>web.php:267-273</code>	<code>NotificationController</code>	<code>SystemNotification</code>	<code>notifications/index.blade.php</code>
AJAX helpers (throttled 120/min)	<code>web.php:213-265</code>	closures + <code>RequisitionController</code> , <code>StockTransferController</code>	—	—

Generated from the codebase: `routes/web.php`, `bootstrap/app.php`, all controllers under `app/Http/Controllers`, models under `app/Models`, services under `app/Services`, middleware under `app/Http/Middleware`, and resources/views.